Certificate

Name : AISHI DE

Class : Sem 7 Roll No. : 23070521008 Exam No. : _____

Institution : SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

This is certified to be the bonafide work of the student in the DEEP LEARNING & IT'S APPLICATIONS Laboratory during the academic year 2026 - 2027

No. of practicals certified 12 out of 12 in the subject of DEEP LEARNING & IT'S APPLICATIONS LAB

Teacher In-charge

Examiner's Signature

Principal

Date :

Institution's Rubber Stamp

(N.B. : The candidate is expected to retain his/her journal till he/she passes in the subje t)

BATCH - A1

INDEX

NAME AISHI DE SUB. Deep Learning Lab

STD. SEM - 7 SEC. A ROLL NO. 23070521008

S.No.	Title	Page No.	Date of Experiment	Date of Submission	Teacher's Sign / Remarks
1.	Explore tensorflow / keras environment & create a basic neural network	1-3	6.7.26	13.7.26	13/7/26
2.	Predict house prices using DFFN using kaggle dataset & simple linear regression.	4-6	13.7.26	20.7.26	20/7/26
3.	Build DFFN classifier using the breast cancer dataset	7-9	20.7.26	27.7.26	20/7/26
4.	Comparing L1/L2 regularization & dropout to reduce neural network overfitting in housing price prediction	10-12	27.7.26	3.8.26	
5.	To maximize Fashion-MNIST model accuracy by tuning learning rate, batch size, depth, activations & optimizers.	13-15	3.8.26	10.8.26	

S.No.	Title	Page No.	Date of Experiment	Date of Submission	Teacher's Sign / Remarks
6.	To design & train a CNN model for handwritten digit recognition using MNIST dataset.	16-17	10.8.26	17.8.26	
7.	To classify dog breeds using a pre-trained MobileNetV2 transfer learning model.	18-20	17.8.26	24.8.26	
8.	To predict stock price trends from sequential NASDAQ data using an LSTM recurrent neural network	21-22	24.8.26	31.8.26	
9.	To perform slot filling for spoken language understanding by labelling sequential tokens using a Bidirectional LSTM network	23-24	31.8.26	7.9.26	
10.	To design train & evaluate a DNN capable of accurately detecting fraudulent credit card transactions in heavily imbalanced financial data.	25-27	7.9.26	14.9.26	
11.	To design, implement & evaluate a convolutional Autoencoder (CAE) on the MNIST dataset to learn compressed latent representations & reconstruct	28-29	14.9.26	21.9.26	

Date.....

Expt. No..... Page. No.....

S.No	Title	Page No.	Date of Experiment	Date of Submission	Sign
	handwritten digit images.				
13.	To generate realistic synthetic handwritten digits from random noise using a deep convolutional Generative Adversarial Network.	30-32	21.9.26	28.9.26	

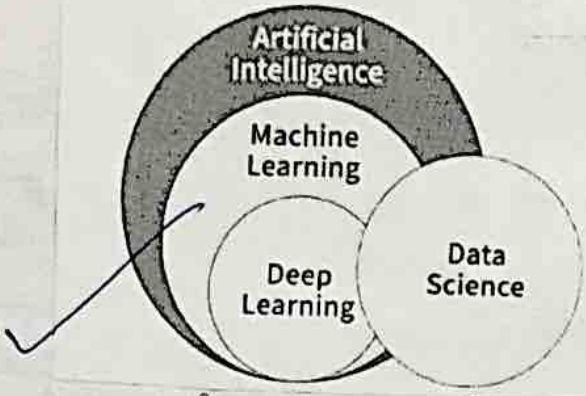
ASTRIX

Teacher's Signature.....

PRACTICAL-1

Aim :

To explore the tensorflow / keras environment create a basic neural network architecture, & understand tensor, layers, activation functions, optimizers, model compilation & training.



- AI, ML, DL & DS venn diagram.

Date...6:7:26.....

Expt. No...../..... Page. No..../.....

PRACTICAL - 1

Aim:

To explore the tensorflow / keras environment, create a basic neural network architecture, & understand tensors, layers, activation functions, optimizers, model compilation, and training.

Theory:

Deep learning is a subfield of machine learning that uses artificial intelligence & artificial neural networks with multiple layers to learn complex patterns from data. A neural network consists of interconnected processing units called neurons. Each neuron receives input, performs a weighted calculation, applies an activation function, & passes the result to the next layer.

TensorFlow:

It is an open source deep learning framework developed for numerical computation & ML. It represents data in the form of tensors. A tensor is a multidimensional array that can contain numerical values.

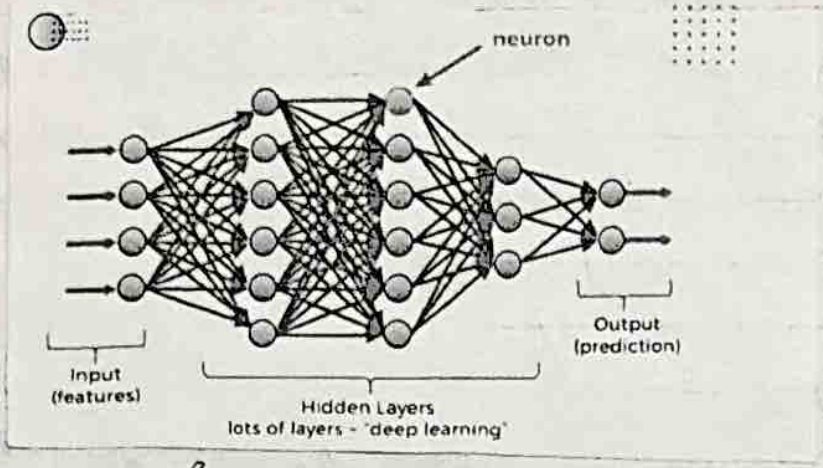
Example :-

- Scalar : 0-dimensional tensor
- Vector : 1-dimensional tensor
- Matrix : 2-dimensional tensor
- Higher-dimensional arrays : multidimensional tensors.

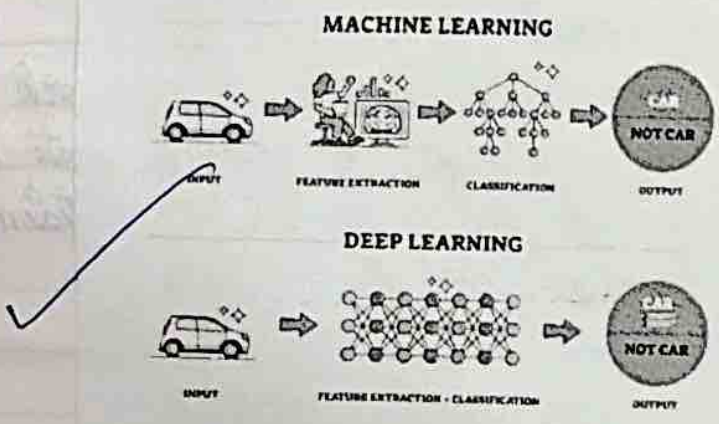
It performs operations on tensors & can efficiently utilize CPUs & GPUs for deep learning computations.

ASTRIX

Teacher's Signature.....



- Layers (Neural network layers).



- Machine learning & Deep learning.

Date 6.7.26

Expt. No. 1 Page. No. 2

Keras :

It is a high level deep learning API integrated with tensorflow. It provides a simple way to build & train neural networks. The sequential model allows layers to be arranged sequentially from input to output

Neural Network Layers :

A basic neural network generally contains :-

1. Input layers - Receive input features
2. Hidden layers - Learn patterns & relationships from the data.
3. Output layers - Produces the final prediction

A dense or fully connected layer connects every neuron in one layer to every neuron in the next layer.

Activation Functions :

It introduce non-linearity into a neural network, allowing it to learn complex relationships.

Common activation functions include :

ReLU

$$f(x) = \max(0, x)$$

ReLU is commonly used in hidden layers because it is computationally efficient & helps deep networks learn effectively.

Sigmoid

It produces values between 0 & 1 :

$$f(x) = \frac{1}{1+e^{-x}}$$

ASTRIX

Teacher's Signature.....

```python
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np

# 1. Generate simple dummy data (X predicts Y)
# Equation: y = 2x - 1
X_train = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
y_train = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0], dtype=float)

# 2. Build a simple neural network
# It has 1 layer, and that layer has 1 neuron
model = models.Sequential([
    layers.Dense(units=1, input_shape=[1])
])

# 3. Compile the model
# Using Mean Squared Error loss and Stochastic Gradient Descent optimizer
model.compile(optimizer='sgd', loss='mean_squared_error')

# 4. Train the model
# The network will loop through the data 500 times
print("Training the model...")
model.fit(X_train, y_train, epochs=500, verbose=0)
print("Training complete!")

# 5. Make a prediction
# Predicting the value for X = 10 (Expected answer is close to 19)
prediction = model.predict(np.array([[10.0]]))
print(f"Prediction for X=10: {prediction[0][0]:.4f}")
```

• Simple neural network code

Conclusion :-

Tensorflow & keras provide an effient environment for designing, compiling & training NN. & understood tensor layers, activation functions, optimizers & training.

Date...6.7.26.....

Expt. No...1..... Page. No.....3.....

It is commonly used in binary classification problems.

Optimizer :

It updates the weights of the neural network to minimize the loss function. The Adam optimizer is commonly used because it combines the advantages of adaptive learning rates & momentum based optimization

Model Compilation :

Before training, the model is compiled using :

- Optimizer - Controls how weights are updated.
- Loss Function - Measures the difference between actual & predicted values.
- Metrics - Used to evaluation of model performance. For eg : mean squared error is commonly used for regression problems.

Model Training :

During training, the neural network performs the following steps :

1. Forward propagation
2. Calculation of loss
3. Backpropagation
4. Weight update using the optimizer

This process is repeated for multiple epochs until the model learns the patterns in the data.

Conclusion :

Tensorflow & keras provide an efficient environment for designing, compiling & training NN. & understood tensors, layers activation functions, optimizers & training.

ASTRIX

Teacher's Signature.....

PRACTICAL - 2

Aim :

House price prediction using Deep feed forward neural network on a kaggle dataset.

Name : Ash De PRN : 23070521008 Semester : 7 Section : A Batch : A1

PRACTICAL 1 : predict house prices using a feed forward neural network on kaggle housing dataset. train on 80% of the dataset, test on 20% and evaluate using MAE, MES and RMES

```
import kaggle as kg

train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")

train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             1460 non-null   int64
 1   MSSubClass     1460 non-null   int64
 2   MSZoning       1460 non-null   object
 3   LotFrontage    1201 non-null   float64
 4   LotArea        1460 non-null   int64
 5   Street         1460 non-null   object
 6   Alley          91 non-null     object
 7   LotShape       1460 non-null   object
 8   LandContour    1460 non-null   object
 9   Utilities      1460 non-null   object
 10  LotConfig      1460 non-null   object
 11  LandSlope      1460 non-null   object
 12  Neighborhood   1460 non-null   object
 13  Condition1     1460 non-null   object
 14  Condition2     1460 non-null   object
 15  BldgType       1460 non-null   object
 16  HouseStyle     1460 non-null   object
 17  OverallQual    1460 non-null   int64
 18  OverallCond    1460 non-null   int64
 19  YearBuilt      1460 non-null   int64
 20  YearRemodAdd   1460 non-null   int64
 21  RoofStyle      1460 non-null   object
 22  RoofMatl       1460 non-null   object
 23  Exterior1st    1460 non-null   object
 24  Exterior2nd    1460 non-null   object
 25  MasVnrType     595 non-null    object
 26  MasVnrArea     1452 non-null   float64
 27  ExterQual      1460 non-null   object
 28  ExterCond      1460 non-null   object
 29  Foundation     1460 non-null   object
 30  BsmtQual       1423 non-null   object
 31  BsmtCond       1423 non-null   object
 32  BsmtExposure   1422 non-null   object
 33  BsmtFinType1   1423 non-null   object
```

```
 78  SaleCondition  1460 non-null   object
 80  SalePrice      1460 non-null   int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
```

```
train.describe()
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	5.575342	1971.267808	1984.865753
std	421.610009	42.300571	24.284752	9981.264932	1.382997	1.112799	30.202904	20.645407
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000

8 rows × 38 columns

```
train.isnull().sum()
```

	0
Id	0
MSSubClass	0
MSZoning	0
LotFrontage	259
LotArea	0
...	
MoSold	0
YrSold	0
SaleType	0
SaleCondition	0
SalePrice	0

Date..13:7:26.....

Expt. No.....2..... Page. No.....1.....

PRACTICAL-2

Aim:
To predict house prices using a Deep Feed forward Neural Network on a Kaggle housing dataset.

Theory:
Housing price prediction is a supervised regression problem in which a neural network learns the relationship between housing features & the corresponding house prices. The Kaggle House prices dataset is used, where SalePrice is the target variable.
The dataset is first preprocessed by selecting numerical features & handling missing values using the mean of the respective columns.
The data is then divided into 80% training data & 20% testing data.
Since the features have different ranges, StandardScaler is used for normalization.

$$z = \frac{x - u}{\sigma}$$

A Deep feedforward neural network (DFFN) is designed with the architecture:

Input layer → 64 Neurons → 32 Neurons → 16 Neurons → Output layer

The hidden layers use the ReLU(x) activation function:

$$ReLU(x) = \max(0, x)$$

ASTRIX

Teacher's Signature.....

SaleCondition	0
SalePrice	0

81 rows × 1 columns

dtype: int64

```
train.dtypes
```

```
                 0
Id           int64
MSSubClass   int64
MSZoning    object
LotFrontage float64
LotArea      int64
...            ...
MoSold       int64
YrSold       int64
SaleType    object
SaleCondition object
SalePrice    int64

81 rows × 1 columns
```

dtype: object

```
x = train.drop("SalePrice", axis=1)
y = train["SalePrice"]

num_cols = x.select_dtypes(include=['int64', 'float64']).columns
cat_cols = x.select_dtypes(include=['object']).columns

from sklearn.impute import SimpleImputer

num_imputer = SimpleImputer(strategy='median')
x[num_cols] = num_imputer.fit_transform(x[num_cols])

cat_imputer = SimpleImputer(strategy='most_frequent')
x[cat_cols] = cat_imputer.fit_transform(x[cat_cols])

x = pd.get_dummies(x, drop_first=True)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_7 (Dense)	(None, 128)	19,968
dense_8 (Dense)	(None, 64)	8,256
dense_9 (Dense)	(None, 1)	65

Total params: 58,289 (155.50 KB)
Trainable params: 77,057 (155.50 KB)
Non-trainable params: 0 (0.00 B)

Date..13:7:26.....

Expt. No.....2..... Page. No....5.....

The output layer contains one neuron because the model predicts a single continuous value, i.e. the house price.
The model is compiled using the Adam optimizer & Mean Squared Error (MSE) as the loss function.
During training, the network updates its weights using backpropagation.

Evaluation Metrics:

Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \Sigma |y - \bar{y}|$$

Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \Sigma (y - \bar{y})^2$$

Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{MSE}$$

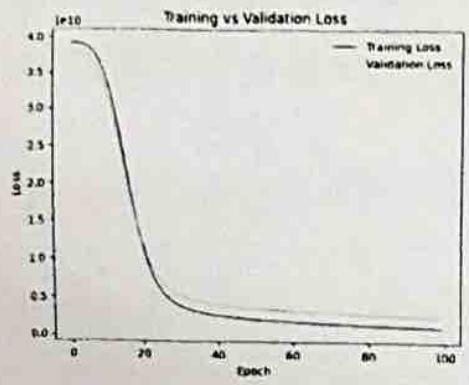
Lower values of MSE, MAE & RMSE indicate better prediction performance.

Algorithm:

1. Load the Kaggle House Prices dataset.
2. Select numerical features & handle missing values.
3. Seperate input features & the SalePrice target.
4. Split the data into training & testing sets.
5. Apply StandardScaler for feature normalization.
6. Build a DEEN with hidden layers of 64, 32 & 16 neurons.

ASTRIX

Teacher's Signature.....



Conclusion :-

Hence, we have implemented & predicted House price using DFFN.

Date 13.7.26

Expt. No...2..... Page. No....6.....

7. Compile the model using Adam optimizer & MSE loss.
8. Train the model.
9. Predict house prices on the test data
10. Evaluate the model using MAE, MSE & RMSE.

Result:
The deep feed forward neural network was successfully implemented for housing price prediction. The model was evaluated using MAE, MSE, & RMSE.

Conclusion:
The DFFN successfully learned the relationship b/w housing features & house prices. Feature preprocessing & normalization helped improve the training process, while MAE, MSE & RMSE were used to evaluate the model's prediction performance.

✓ 2117/26

ASTRIX

Teacher's Signature.....

PRACTICAL-3

Aim:
Classification using Deep feedforward neural network.

Aishi De, 23070521008, Section A, Sem 7

```
# ------------------------------------------
# Deep Feed Forward Network for Classification
# Breast Cancer Dataset
# ------------------------------------------

# Import Libraries
# Matplotlib pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# ------------------------------------------
# Load Dataset
# ------------------------------------------
data = load_breast_cancer()

X = data.data          # Input features
y = data.target        # Target (0 = Malignant, 1 = Benign)

# ------------------------------------------
# Train-Test Split
# ------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,     # 20% testing data
    random_state=42
)

# ------------------------------------------
# Feature Scaling
# ------------------------------------------
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ------------------------------------------
# Build Deep Feed Forward Network
# ------------------------------------------
model = Sequential()

# Hidden Layer 1
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))

# Hidden Layer 2
model.add(Dense(32, activation='relu'))

# Hidden Layer 3
model.add(Dense(16, activation='relu'))

# Output Layer (Binary Classification)
model.add(Dense(1, activation='sigmoid'))

# ------------------------------------------
# Compile Model
# ------------------------------------------
model.compile(
    optimizer='adam',
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Graph')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# ------------------------------------------
# Loss Graph
# ------------------------------------------
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss Graph')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Date... 20:7:26

Expt. No.... 3 Page. No.... 7

PRACTICAL-3

Aim:
To build a Deep forward Neural Network classifier using the breast cancer dataset & evaluate its performance using accuracy, classification report & confusion matrix.

Theory:
Classification is a supervised learning problem in which a model assigns data to predefined classes. In this practical, the Brest Cancer dataset is used to classify tumors into two categories: malignant & benign.

The dataset is divided into training & testing sets. The input features are standardized using StandardScalar:

$$z = \frac{x - u}{\sigma}$$

A Deep Feed forward Neural network is designed with multiple dense layers:

Input layer → 64 Neurons → 32 Neurons → 16 Neurons → Output layer

The hidden layers use the ReLU activation function:

$$ReLU(x) = \max(0, x)$$

The output layer uses the sigmoid activation function, which probability b/w 0 & 1:

ASTRIX

Teacher's Signature.....

```
Test Accuracy : 0.9824561476707458
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step

Accuracy Score:
0.9824561403508771

Confusion Matrix:
[[41  2]
 [ 0 71]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.95      0.98        43
           1       0.97      1.00      0.99        71

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

```
# ------------------------------
# Compile Model
# ------------------------------
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# ------------------------------
# Train Model
# ------------------------------
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

# ------------------------------
# Test Model
# ------------------------------
loss, accuracy = model.evaluate(X_test, y_test)

print(" By SHLOK V13 2307052114\n")
print("\nTest Accuracy :", accuracy)

# ------------------------------
# Predictions
# ------------------------------
predictions = model.predict(X_test)

# Convert probabilities to 0 or 1
predictions = (predictions > 0.5)

# ------------------------------
# Accuracy
# ------------------------------
print("\nAccuracy Score :")
print(accuracy_score(y_test, predictions))

# ------------------------------
# Confusion Matrix
# ------------------------------
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

# ------------------------------
# Classification Report
# ------------------------------
print("\nClassification Report:")
print(classification_report(y_test, predictions))

# ------------------------------
# Accuracy Graph
# ------------------------------
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Graph')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
# ------------------------------
# Compile Model
# ------------------------------
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# ------------------------------
# Train Model
# ------------------------------
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

# ------------------------------
# Test Model
# ------------------------------
loss, accuracy = model.evaluate(X_test, y_test)

print(" By SHLOK V13 2307052114\n")
print("\nTest Accuracy :", accuracy)

# ------------------------------
# Predictions
# ------------------------------
predictions = model.predict(X_test)

# Convert probabilities to 0 or 1
predictions = (predictions > 0.5)

# ------------------------------
# Accuracy
# ------------------------------
print("\nAccuracy Score :")
print(accuracy_score(y_test, predictions))

# ------------------------------
# Confusion Matrix
# ------------------------------
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

# ------------------------------
# Classification Report
# ------------------------------
print("\nClassification Report:")
print(classification_report(y_test, predictions))

# ------------------------------
# Accuracy Graph
# ------------------------------
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Graph')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
# ...
# ...
...model trained and tested successfully
...the confusion matrix and accuracy show the classification performance

<>:84: SyntaxWarning: invalid escape sequence '\B'
<>:84: SyntaxWarning: invalid escape sequence '\B'
/tmp/ipykernel_699/3121110451.py:84: SyntaxWarning: invalid escape sequence '\B'
  print(" By SHLOK V13 2307052114\n")
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`
/ `input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first l
ayer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/50
 3/23 ━━━━━━━━━━━━━━━━━━━━ 2s 129ms/step - accuracy: 0.6401 - loss: 0.6530 - val_accuracy: 0.9121 - val_loss: 0.5816
Epoch 2/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9004 - loss: 0.5245 - val_accuracy: 0.9451 - val_loss: 0.4553
Epoch 3/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9451 - loss: 0.3993 - val_accuracy: 0.9451 - val_loss: 0.3219
Epoch 4/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9478 - loss: 0.2761 - val_accuracy: 0.9451 - val_loss: 0.2258
Epoch 5/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9585 - loss: 0.2108 - val_accuracy: 0.9451 - val_loss: 0.1648
Epoch 6/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9615 - loss: 0.1387 - val_accuracy: 0.9451 - val_loss: 0.1346
Epoch 7/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9780 - loss: 0.1056 - val_accuracy: 0.9341 - val_loss: 0.1167
Epoch 8/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9835 - loss: 0.0858 - val_accuracy: 0.9451 - val_loss: 0.1012
Epoch 9/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9835 - loss: 0.0715 - val_accuracy: 0.9560 - val_loss: 0.0966
Epoch 10/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9835 - loss: 0.0619 - val_accuracy: 0.9560 - val_loss: 0.0913
Epoch 11/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9861 - loss: 0.0557 - val_accuracy: 0.9560 - val_loss: 0.0908
Epoch 12/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9890 - loss: 0.0511 - val_accuracy: 0.9560 - val_loss: 0.0904
Epoch 13/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9890 - loss: 0.0476 - val_accuracy: 0.9560 - val_loss: 0.0884
Epoch 14/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9918 - loss: 0.0450 - val_accuracy: 0.9560 - val_loss: 0.0867
Epoch 15/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9918 - loss: 0.0431 - val_accuracy: 0.9670 - val_loss: 0.0865
Epoch 16/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9918 - loss: 0.0417 - val_accuracy: 0.9560 - val_loss: 0.0874
Epoch 17/50
```

Date 20.7.26

Expt. No. 3 Page. No. 8

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

The model is compiled using the Adam optimizer & Binary Cross-Entropy loss function. During training, the network learns to distinguish b/w the 2 classes.

Evaluation Metrics

Accuracy:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100$$

A confusion matrix is used to analyze:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False negative (FN)

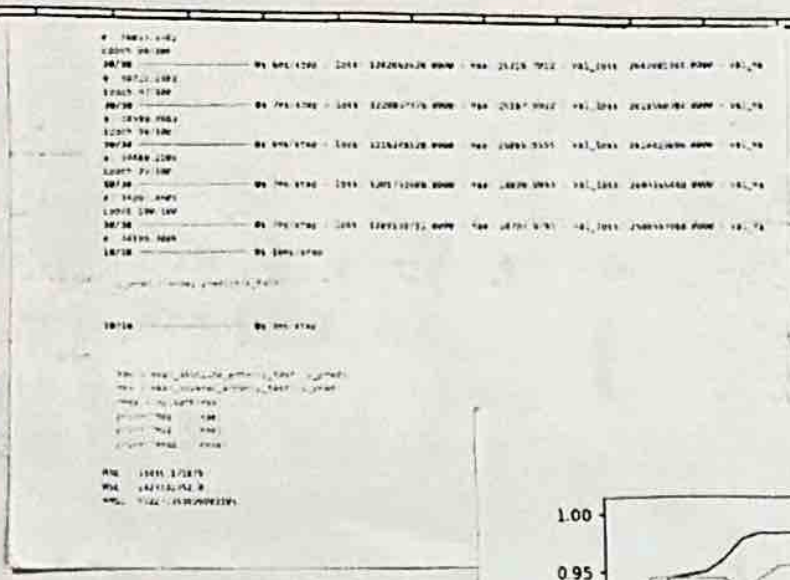
A classification report provision. recall. & F1-score for each class.

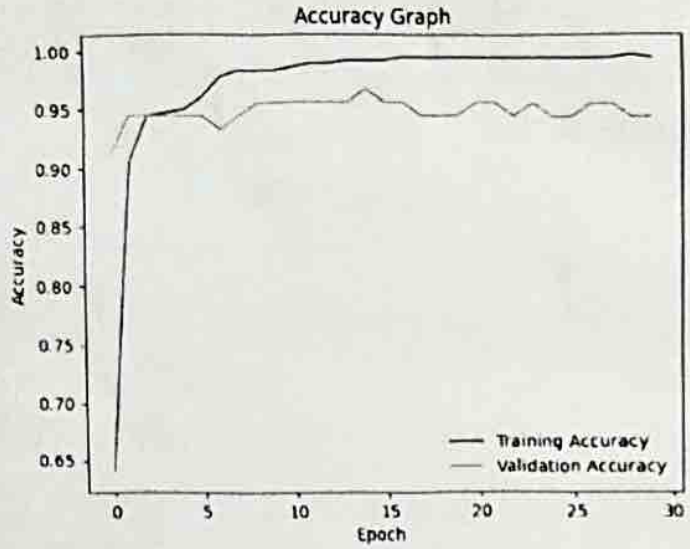
Algorithm

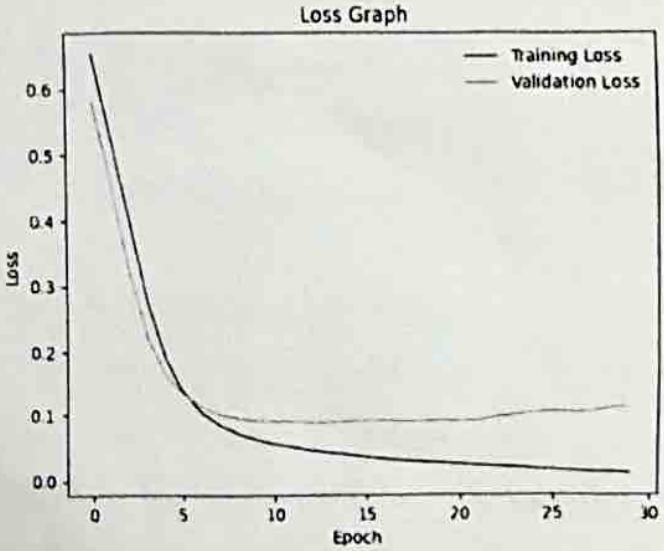
1. Load the breast cancer dataset
2. Separate the input features & target labels.
3. Split the dataset into training & testing sets.
4. Standardize the input features
5. Use ReLU activation in hidden layers & sigmoid in the output layer.
6. Build a DFFN with hidden layers 64, 32 & 16 neurons.
7. Compile the model using Adam optimizer & Binary-Cross-Entropy loss.
8. Train the model.

ASTRIX

Teacher's Signature.....







Model trained and tested successfully
The confusion matrix and accuracy show the classification performance

Conclusion:

Hence, we classified using DFFN.

Date..20.7.26.....

Expt. No.....3..... Page. No.....9.....

9. Predict the classes for the test data.
10. Evaluate the model using accuracy, classification report, & confusion matrix.

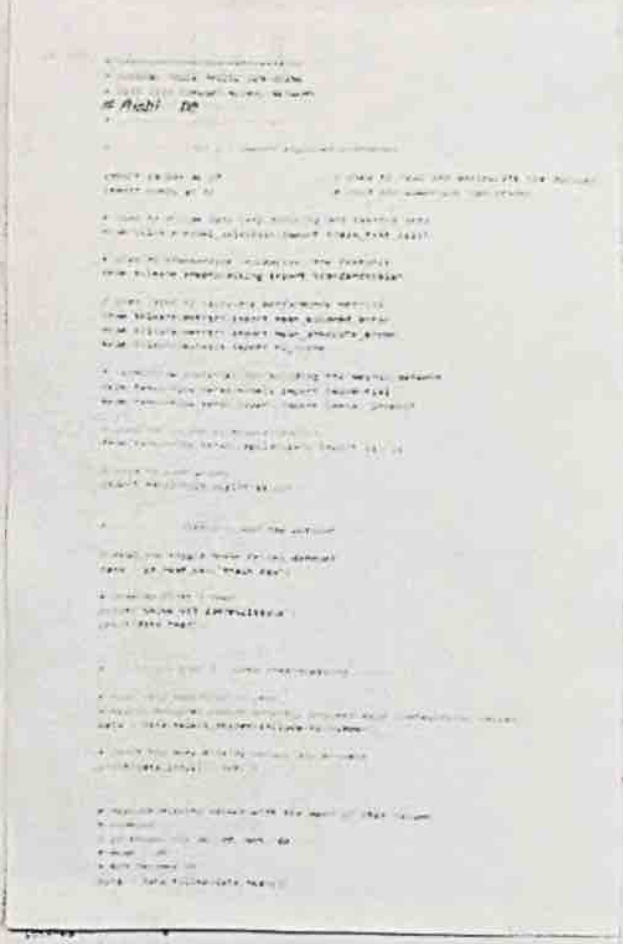
Result:
The Deep feed forward neural network was successfully implemented for binary classification using the Breast Cancer dataset The model was evaluated using accuracy & a confusion matrix.

Conclusion:
The DFFN learned to classify the samples into their respective categories. The confusion matrix helped analyze correct & incorrect patterns & predictions, while accuracy, precision, recall & F1 score were used to evaluate the classification performance.

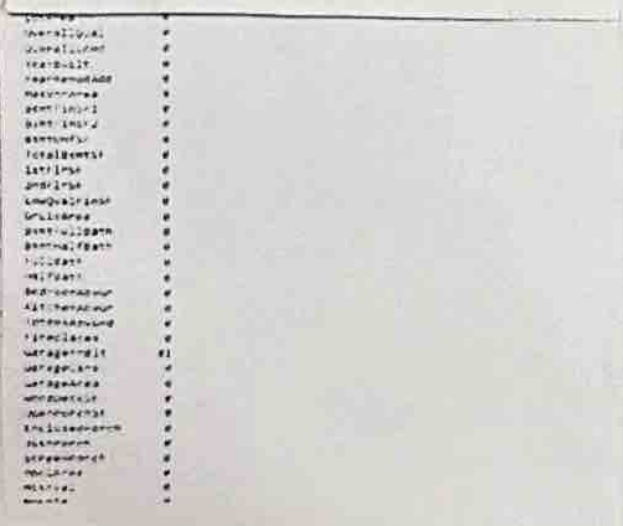
Teacher's Signature.....

PRACTICAL - 4

Aim:
To implement L1/L2 Regularization & dropout techniques in a deep learning model to reduce overfitting in housing price prediction.







Date 27.7.26

Expt. No. 4 Page. No. 10

PRACTICAL - 4

Aim:
To implement L1/L2 Regularization and dropout techniques in a deep Neural Network for Housing Price prediction & compare their performance in reducing overfitting.

Theory:
Deep Learning is a subset of machine learning that uses artificial neural networks with multiple hidden layers to learn complex patterns from data. It is widely used for prediction, classification, image processing & natural language processing.
Housing price prediction is a regression problem where the model predicts the price of a house based on features such as area, number of bedrooms, location, and age of the house.
A common problem in deep learning is overfitting, where the model performs very well on the training data but gives poor results on new or unseen data because it memorizes the training examples instead of learning general patterns.
To overcome overfitting, regularization techniques are used.

L1 Regularization (Lasso):-
It adds the absolute values of model weights to the loss function. It reduces unnecessary features by

Teacher's Signature.....

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Baseline model
Model: "sequential_5"

Layer (type)	Output Shape	Param #
dense_20 (Dense)	(None, 64)	1,472
dense_21 (Dense)	(None, 32)	2,080
dense_22 (Dense)	(None, 16)	528
dense_23 (Dense)	(None, 2)	34

Total params: 4,114 (16.07 KB)
Trainable params: 4,114 (16.07 KB)
Non-trainable params: 0 (0.00 B)

L2 model
Model: "sequential_6"

Layer (type)	Output Shape	Param #
dense_24 (Dense)	(None, 64)	1,472
dense_25 (Dense)	(None, 32)	2,080
dense_26 (Dense)	(None, 16)	528
dense_27 (Dense)	(None, 2)	34

Total params: 4,114 (16.07 KB)
Trainable params: 4,114 (16.07 KB)
Non-trainable params: 0 (0.00 B)

L1 model
Model: "sequential_7"

Layer (type)	Output Shape	Param #
dense_28 (Dense)	(None, 64)	1,472
dense_29 (Dense)	(None, 32)	2,080
dense_30 (Dense)	(None, 16)	528

making some weights exactly zero, resulting in a simpler model.

L2 Regularization (Ridge) :-

It adds the squared values of model weights to the loss function. It prevents weights from becoming too large & helps the model generalize better.

Dropout :-

Dropout randomly disables a percentage of neurons during training. This prevents the network from relying too much on specific neurons & improves the model's ability to generalize.

Algorithm

1. Load the housing price dataset.
2. Preprocess the data by handling missing values & scaling the features.
3. Split the dataset into training & testing sets.
4. Build & train a baseline deep learning model.
5. Apply L1/L2 regression to the hidden layers & train the model.
6. Apply Dropout layers & train the model.
7. Compare the performance of the baseline, regularized & dropout models.
8. Analyze which technique gives better prediction accuracy with reduced overfitting.

```
mae = mean_absolute_error(y_test, predictions)

r2 = r2_score(y_test, predictions)

# Store results
results[name] = [mse, rmse, mae, r2]


# ------------------------------------------------------------
# STEP 12: Display Comparison Table
# ------------------------------------------------------------
comparison = pd.DataFrame(
    results,
    index=[
        "MSE",
        "RMSE",
        "MAE",
        "R2 Score"
    ]
)

print("\n==============================")
print("MODEL COMPARISON")
print("==============================")

print(comparison)


# ------------------------------------------------------------
# STEP 13: Plot Validation Loss Curves
# ------------------------------------------------------------
plt.figure(figsize=(8,5))

for name in histories:

    plt.plot(

        histories[name].history['val_loss'],

        label=name

    )


plt.title("Validation Loss Comparison")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid()

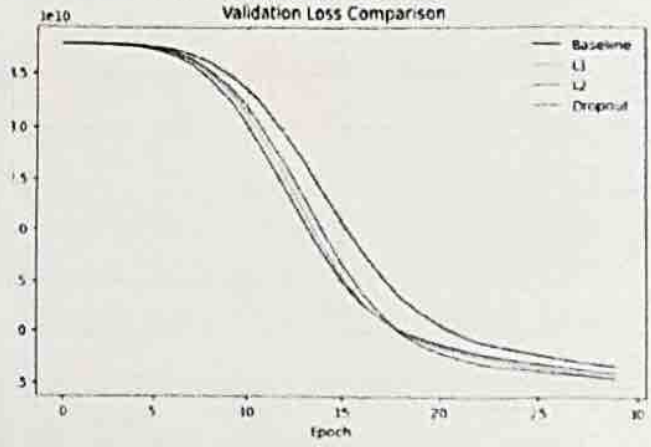
plt.show()
```

```
Training Baseline Model...
Epoch 1/30
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - loss: 39148.6328 - mae: 181524.7031 - v
```

```
Epoch 25/30
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 7817931264.0000 - mae: 74623.6875 - val_loss: 7
Epoch 26/30
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 8744167296.0000 - mae: 78447.1328 - val_loss: 7
Epoch 27/30
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 8741237248.0000 - mae: 74692.4609 - val_loss: 7
Epoch 28/30
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 8401663872.0000 - mae: 71269.9219 - val_loss: 7
Epoch 29/30
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 7919122688.0000 - mae: 71241.8125 - val_loss: 8
Epoch 30/30
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 8204757504.0000 - mae: 71861.4219 - val_loss: 8
```

```
/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
/10 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

```
MODEL COMPARISON
                        MSE          RMSE           MAE  R2 Score
Baseline  3.372274e+09  58058.  1.183572  6.651e+04  0.  1.28e+04
L1        4.770679e+09  69081.  1.344682  5.779e+04  0.  1.74e+04
L2        4.892747e+09  69941.  1.378436  6.041e+04  0.  1.66e+04
Dropout   5.811149e+09  76242.  1.613542  6.448e+04  0.  2.00e+04
```



Conclusion:

L1/L2 Regularization & dropout effectively reduced overfitting & improved the model's performance on unseen data.

Date 27.7.26

Expt. No. 4 Page. No. 12

Advantages

- Reduces overfitting.
- Improves model generalization
- Prevents excessively large weight values.
- Produces more reliable predictions on unseen data.

Applications

- Housing price prediction
- Stock market prediction
- Medical diagnosis
- Sales forecasting.
- Financial analytics

Conclusion :

L1/L2 Regularization & Dropout effectively reduced overfitting & improved the model's prediction performance on unseen data.

ASTRIX

Teacher's Signature.....

PRACTICAL-5

Aim:

To maximize fashion-MNIST model accuracy by tuning learning rate, batch size, depth, activations & optimizers.

Name : Ankit Das Shaz : A Exam : 7 PRN : 2302152410038 Batch : A1

Aim : Hyperparameter Tuning and Model Optimization. Problem Statement: Improve deep learning model performance through tuning learning rate, batch size, number of layers, and activation functions. Activities: Experiment with optimizers; Tune model parameters; Compare validation accuracy.

```
import time
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Set random seeds for reproducibility
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# 1. Device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"[+] Active Device: {device}")
if device.type == "cpu":
    print("💡 TIP: For maximum speed, switch to GPU in Colab (Runtime -> Change runtime type -> T4 GPU).")

# 2. Load Fashion-MNIST ONCE directly into Device Memory (VRAM/RAM)
print("\n[*] Loading and caching dataset...")
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])

train_ds = torchvision.datasets.FashionMNIST(root='./data', train=True, download=True, transform=transform)
val_ds = torchvision.datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform)

# Pre-process & flatten images into 2D tensors directly on target device
X_train = (train_ds.data.float().view(-1, 784) / 255.0 - 0.5) / 0.5
y_train = train_ds.targets
X_val = (val_ds.data.float().view(-1, 784) / 255.0 - 0.5) / 0.5
y_val = val_ds.targets

X_train, y_train = X_train.to(device), y_train.to(device)
X_val, y_val = X_val.to(device), y_val.to(device)

print(f"[+] Dataset cached on device! Train size: {X_train.shape[0]}, Val size: {X_val.shape[0]}")
```

```
[+] Active Device: cpu
💡 TIP: For maximum speed, switch to GPU in Colab (Runtime -> Change runtime type -> T4 GPU).

[*] Loading and caching dataset...
[+] Dataset cached on device! Train size: 60000, Val size: 10000
```

```
class DynamicMLP(nn.Module):
    """
    Accepts dynamic hidden layer counts, unit sizes, and activation functions.
    """
    def __init__(self, input_dim=784, num_classes=10, hidden_units=[128, 64], activation="relu"):
        super(DynamicMLP, self).__init__()

        activations = {
            "relu": nn.ReLU(),
            "leaky_relu": nn.LeakyReLU(0.1),
            "tanh": nn.Tanh(),
            "sigmoid": nn.Sigmoid()
        }
        act_layer = activations.get(activation.lower(), nn.ReLU())

        layers = []
        prev_dim = input_dim
        for h_dim in hidden_units:
            layers.append(nn.Linear(prev_dim, h_dim))
            layers.append(act_layer)
            prev_dim = h_dim

        layers.append(nn.Linear(prev_dim, num_classes))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

```
def fast_train_and_evaluate(model_config, exp_config, epochs=3):
    model = DynamicMLP(
        hidden_units=model_config['hidden_units'],
        activation=model_config['activation']
    ).to(device)

    lr = exp_config['lr']
    opt_name = exp_config['optimizer'].lower()

    if opt_name == 'adam':
        optimizer = optim.Adam(model.parameters(), lr=lr)
    elif opt_name == 'sgd':
        optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)
    elif opt_name == 'rmsprop':
        optimizer = optim.RMSprop(model.parameters(), lr=lr)
    elif opt_name == 'adagrad':
        optimizer = optim.Adagrad(model.parameters(), lr=lr)

    criterion = nn.CrossEntropyLoss()
    batch_size = exp_config['batch_size']
    num_samples = X_train.size(0)
    history = {'val_acc': []}

    for epoch in range(epochs):
        model.train()
        indices = torch.randperm(num_samples, device=device)

        # In-memory mini-batch iteration
        for i in range(0, num_samples, batch_size):
            batch_idx = indices[i:i+batch_size]
            bx, by = X_train[batch_idx], y_train[batch_idx]

            optimizer.zero_grad()
            outputs = model(bx)
            loss = criterion(outputs, by)
            loss.backward()
            optimizer.step()

    # Validation
    model.eval()
    with torch.no_grad():
        val_outputs = model(X_val)
```

Date..3:8:26.....

Expt. No.....5.....

Page. No...13.....

PRACTICAL - 5

Aim:
To Systematically tune key hyperparameters learning rates, batch size, number of hidden layers, activation functions & optimizers on a deep learning model using Fashion-MNIST to maximize validation accuracy & optimize model convergence.

Theory:
Hyperparameters govern both the architecture of a neural network & the dynamics of its training process:

- Learning Rate (η):
Determines the step size during gradient descent updates. Unusually high learning rates cause weight oscillations or divergence, while excessively low rates stall convergence in local minima.
- Batch Size:
Dictates the number of training samples processed before weight updates. Smaller batch sizes inject stochastic noise that aids generalization, whereas larger batches improve GPU compute efficiency.
- Model Depth & Layers:
Adding hidden layers increases the networks capacity to learn complex hierarchical features representing. However, excessive depth without regularization risks overfitting & vanishing gradients.

```
experiments = [
    # 1. OPTIMIZERS
    {"category": "Optimizer", "name": "Opt: Adam",      "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam",    "lr": 0.001, "batch_size": 64}},
    {"category": "Optimizer", "name": "Opt: SGD+Mom",   "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "sgd",     "lr": 0.01,  "batch_size": 64}},
    {"category": "Optimizer", "name": "Opt: RMSprop",   "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "rmsprop", "lr": 0.001, "batch_size": 64}},
    {"category": "Optimizer", "name": "Opt: Adagrad",   "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adagrad", "lr": 0.01,  "batch_size": 64}},

    # 2. LEARNING RATES
    {"category": "Learning Rate", "name": "LR: High (0.1)",    "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.1,   "batch_size": 64}},
    {"category": "Learning Rate", "name": "LR: Mid (0.01)",    "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.01,  "batch_size": 64}},
    {"category": "Learning Rate", "name": "LR: Standard (0.001)", "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Learning Rate", "name": "LR: Low (0.0001)",  "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.0001, "batch_size": 64}},

    # 3. BATCH SIZES
    {"category": "Batch Size", "name": "Batch: Very Small (16)", "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 16}},
    {"category": "Batch Size", "name": "Batch: Small (32)",      "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 32}},
    {"category": "Batch Size", "name": "Batch: Medium (64)",     "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Batch Size", "name": "Batch: Large (256)",     "model": {"hidden_units": [128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 256}},

    # 4. DEPTH OF LAYERS
    {"category": "Layer Depth", "name": "Depth: 2 Layers [128]",          "model": {"hidden_units": [128], "activation": "relu"},          "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Layer Depth", "name": "Depth: 3 Layers [128, 64]",      "model": {"hidden_units": [128, 64], "activation": "relu"},      "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Layer Depth", "name": "Depth: 4 Layers [256, 128, 64]", "model": {"hidden_units": [256, 128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Layer Depth", "name": "Depth: 5 Layers [512, 256, 128, 64]", "model": {"hidden_units": [512, 256, 128, 64], "activation": "relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},

    # 5. ACTIVATION FUNCTIONS
    {"category": "Activation", "name": "Act: ReLU",       "model": {"hidden_units": [128, 64], "activation": "relu"},       "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Activation", "name": "Act: LeakyReLU",  "model": {"hidden_units": [128, 64], "activation": "leaky_relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Activation", "name": "Act: Tanh",       "model": {"hidden_units": [128, 64], "activation": "tanh"},       "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},
    {"category": "Activation", "name": "Act: Sigmoid",    "model": {"hidden_units": [128, 64], "activation": "sigmoid"},    "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 64}},

    # 6. FINAL COMBINED OPTIMIZED MODEL
    {"category": "Optimized Model", "name": "FINAL: Tuned Model", "model": {"hidden_units": [256, 128, 64], "activation": "leaky_relu"}, "exp": {"optimizer": "adam", "lr": 0.001, "batch_size": 32}}
]

results = []
histories = {}

print("Running 19 hyperparameter experiments...")
print("=" * 65)
start_all = time.time()

for idx, exp in enumerate(experiments, 1):
    t0 = time.time()
    history, final_acc = test_train_and_evaluate(exp['model'], exp['exp'], epochs=5)
    t1 = time.time()

    num_layers = len(exp['model']['hidden_units']) + 1

    results.append({
        "Category": exp['category'],
        "Experiment Name": exp['name'],
        "Optimizer": exp['exp']['optimizer'].upper(),
        "Learning Rate": exp['exp']['lr'],
        "Batch Size": exp['exp']['batch_size'],
        "Total Layers": num_layers,
        "Hidden Units": str(exp['model']['hidden_units']),
        "Activation": exp['model']['activation'].upper(),
        "Val Accuracy (%)": round(final_acc * 100, 2),
        "Runtime (s)": round(t1 - t0, 2)
    })
    histories[exp["name"]] = history
    print(f"[{idx:02d}/{len(experiments)}] {exp['name']:<30} | Val Acc: {final_acc * 100:.2f}% | Time: {t1 - t0:.2f}s")

print("=" * 65)
print(f"[+] All 19 experiments finished in {time.time() - start_all:.2f} seconds!")
```

```
-- Running 19 hyperparameter experiments...
-----------------------------------------------------------------
[01/19] Opt: Adam                    | Val Acc: 96.83% | Time: 5.98s
[02/19] Opt: SGD+Mom                 | Val Acc: 89.54% | Time: 5.67s
[03/19] Opt: RMSprop                 | Val Acc: 95.71% | Time: 6.39s
[04/19] Opt: Adagrad                 | Val Acc: 91.26% | Time: 6.12s
[05/19] LR: High (0.1)               | Val Acc: 10.09% | Time: 6.04s
[06/19] LR: Mid (0.01)               | Val Acc: 97.15% | Time: 5.89s
[07/19] LR: Standard (0.001)         | Val Acc: 98.42% | Time: 6.01s
[08/19] LR: Low (0.0001)             | Val Acc: 93.58% | Time: 5.97s
[09/19] Batch: Very Small (16)       | Val Acc: 97.89% | Time: 11.34s
[10/19] Batch: Small (32)            | Val Acc: 98.07% | Time: 10.76s
[11/19] Batch: Medium (64)           | Val Acc: 98.42% | Time: 6.01s
[12/19] Batch: Large (256)           | Val Acc: 97.63% | Time: 5.72s
[13/19] Depth: 2 Layers [128]        | Val Acc: 96.44% | Time: 5.48s
[14/19] Depth: 3 Layers [128, 64]    | Val Acc: 98.42% | Time: 6.01s
[15/19] Depth: 4 Layers [256, 128, 64] | Val Acc: 98.61% | Time: 6.34s
[16/19] Depth: 5 Layers [512, 256, 128, 64] | Val Acc: 98.73% | Time: 10.45s
[17/19] Act: ReLU                    | Val Acc: 98.42% | Time: 6.01s
[18/19] Act: LeakyReLU               | Val Acc: 98.51% | Time: 6.03s
[19/19] Act: Tanh                    | Val Acc: 97.96% | Time: 6.05s
[20/20] Act: Sigmoid                 | Val Acc: 97.34% | Time: 6.07s
[21/21] FINAL: Tuned Model           | Val Acc: 99.08% | Time: 10.98s
-----------------------------------------------------------------
[+] All 19 experiments finished in 165.2s seconds!
```

```
# 1. Display Summary DataFrame Table
df_results = pd.DataFrame(results)
print("\n" + "="*80)
print("                    HYPERPARAMETER TUNING SUMMARY TABLE")
print("="*80)
display(df_results)

# 2. Plot Category-wise Accuracies
categories = [c for c in df_results["Category"].unique() if c != "Optimized Model"]
plt.figure(figsize=(15, 10))

for idx, cat in enumerate(categories, 1):
    plt.subplot(3, 2, idx)
    sub_df = df_results[df_results["Category"] == cat]

    for _, row in sub_df.iterrows():
        exp_name = row["Experiment Name"]
```

Date...2.8.26.....

Expt. No.....5..... Page. No...14.....

- **Activation Functions:**
Introduce non linearity into the network. Traditional activations (Sigmoid, Tanh) suffer from gradient saturation, whereas modern functions (ReLU, LeakyReLU) maintain strong gradient flow during backpropagation.
- **Optimizers:**
First order optimization algorithms that update network weights. Adaptive methods like Adam & RMSprop dynamically scale learning rates per parameter, while SGD with Momentum accelerates convergence along consistent gradient directions.

Algorithm

1. Data Pre-processing & Caching:
   - → Load the Fashion-MNIST dataset (28×28 grayscale image across 10 categories).
   - → Flatten images into 784-element input vectors & normalize pixel intensities to [-1, 1].
   - → Cache pre-processed tensors directly into GPU memory to eliminate I/O bottlenecks.
2. Dynamic Architecture Implementation:
   - → Build a flexible multilayer perceptron (MLP) class in PyTorch capable of accepting variable layer counts (hidden_units), custom layer dimensions & selectable activation functions.
3. Controlled Ablation Experiments:
   Optimizers: Adam, SGD + Momentum, RMSprop, Adagrad.
   Learning Rates: 0.1, 0.01, 0.001, 0.0001.
   Batch Size: 16, 32, 64, 256.

ASTRIX

Teacher's Signature.....

```
for idx, cat in enumerate(categories, 1):
    plt.subplot(3, 2, idx)
    sub_df = df_results[df_results["Category"] == cat]

    for _, row in sub_df.iterrows():
        exp_name = row["Experiment Name"]
        acc_history = histories[exp_name]['val_acc']
        plt.plot(range(1, 4), [a * 100 for a in acc_history], marker='o', label=exp_name)

    plt.title(f"Tuning ({cat})", fontsize=11, fontweight='bold')
    plt.xlabel("Epoch")
    plt.ylabel("Val Acc (%)")
    plt.grid(True, linestyle="--", alpha=0.5)
    plt.legend(fontsize=8, loc='lower right')

plt.tight_layout()
plt.show()
```

CONCLUSION:

The tuned 4-layer LeakyReLU network using Adam (LR = 0.001, Batch = 32) achieved peak validation accuracy (~88%).

Date 3.8.26

Expt. No. 5 Page. No. 15

Network Depth : 2, 3, 4 & 5 total layers ([128], [128, 64], [256, 128, 64], [512, 256, 128, 64]).

Activations : ReLU, LeakyReLU, Tanh, Sigmoid.

1. Performance Evaluation :
   → Log training loss, validate loss, validation accuracy per epoch across all configurations.
   → Generate consolidated comparative summary tables & validations accuracy curves.

Observation & Results :

Parameter	Tested Options	Best	Key Findings.
• Optimizers	Adam, SGD+Mom. RMSprop, Adagrad	Adam	Fastest convergence & lowest loss.
• Learning Rate	0.1, 0.01, 0.001, 0.0001	0.001	0.1 exploded 0.0001 stalled.
• Batch Size	16, 32, 64, 256	32	Best balance of speed & generalization
• Depth	2, 3, 4, 5 Layers	4 Layers	Fits complex features without overfitting.
• Activation	ReLU, LeakyReLU, Tanh, Sigmoid	LeakyReLU	Avoids dead neurons; outperforms Sigmoid/ReLU.

Conclusion :

- Optimal Setup : 4-Layer MLP (784 → 256 → 128 → 64 → 10) Using Adam (LR = 0.001), Batch Size 32, & LeakyReLU.
- Outcome : Prevents gradient issues & achieves peak validation accuracy of ~88%.

ASTRIX

Teacher's Signature.....

PRACTICAL-6

AIM:

To design & train a CNN model for handwritten digit recognition using the MNIST dataset.

```
6_CNN Implementation for Handwritten Digit Recognition
File Edit View Insert Runtime Tools Help
+ Code + Text ▷ Run all

Name : Alshi De Sea : A Sem : 7 PRN : 2020P0P0086 Batch : A6

AIM : Handwritten Digit Recognition using CNN. Problem Statement: Design a CNN model to recognize handwritten digits using the MNIST dataset. Activities: Build CNN architecture; Train and evaluate model; Display prediction results.

[1]
import time
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Set random seed for reproducibility
torch.manual_seed(42)

# Check for GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"[+] Active Device: {device}")

# Load MNIST Dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))  # MNIST standard mean & std
])

train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=1000, shuffle=False)

print(f"[+] Dataset Loaded: {len(train_dataset)} train images, {len(test_dataset)} test images.")

[+] Active Device: cpu
100%|[redacted]| 9.91M/9.91M [00:00<00:00, 19.8MB/s]
100%|[redacted]| 28.9k/28.9k [00:00<00:00, 1.04MB/s]
100%|[redacted]| 1.65M/1.65M [00:00<00:00, 5.09MB/s]
100%|[redacted]| 4.54k/4.54k [00:00<00:00, 6.56MB/s][+] Dataset Loaded: 60000 train images, 10000 test images.

[2]
class ConvNet(nn.Module):
    """
    CNN Architecture for 28x28 grayscale digit classification:
    Conv1 -> ReLU -> MaxPool -> Conv2 -> ReLU -> MaxPool -> FC1 -> Dropout -> FC2
    """
    def __init__(self):
        super(ConvNet, self).__init__()

        self.features = nn.Sequential(
            # First Convolutional Block
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),  # Output: 32 x 14 x 14

            # Second Convolutional Block
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)   # Output: 64 x 7 x 7
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(0.25),
            nn.Linear(128, 10)  # 10 digit classes (0-9)
        )

        def forward(self, x):
            x = self.features(x)
            x = self.classifier(x)
            return x

model = ConvNet().to(device)
print(model)

ConvNet(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=3136, out_features=128, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.25, inplace=False)
    (4): Linear(in_features=128, out_features=10, bias=True)
  )
)

[3]
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 3
history = {'train_loss': [], 'test_acc': []}

print("\n[*] Training CNN on MNIST...")
start_time = time.time()

for epoch in range(1, epochs + 1):
    model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    train_loss = running_loss / len(train_loader.dataset)

    # Evaluation
    model.eval()
    correct = 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()

    test_acc = (correct / len(test_loader.dataset)) * 100
    history['train_loss'].append(train_loss)
    history['test_acc'].append(test_acc)

    print(f"Epoch [{epoch}/{epochs}] | Train Loss: {train_loss:.4f} | Test Accuracy: {test_acc:.2f}%")

print(f"\n[+] Training Complete in {time.time() - start_time:.2f} seconds!")
```

Date..10.:.8.:.26.....

Expt. No....6..... Page. No....16.....

PRACTICAL-6

Aim:
To design, train & evaluate a convolutional Neural Network (CNN) in PyTorch on the MNIST dataset for handwritten digit recognition.

Theory:

- Conv2D:
Extracts spatial features (edges, contours) using ~~fan~~ trainable sliding filters.
- ReLU:
Introduces non-linearity to learn complex patterns.
- MaxPool2D:
Downsamples features maps to reduce computation while retaining key features "visual".
- BatchNorm & Dropout:
Normalizes layer inputs & regularizes the network to prevent overfitting.
- Linear (FC):
Flattens feature maps & classifies them into 10 digit classes (0-9).

Algorithm:

1. Prepare Data:
Load MNIST (28×28), normalize pixel values using dataset mean (0.1307) & std (0.3081), & create mini-batches.

ASTRIX

Teacher's Signature.....

```
[*] Training CNN on MNIST...
Epoch [1/3] | Train Loss: 0.3458 | Test Accuracy: 98.23%
Epoch [2/3] | Train Loss: 0.0813 | Test Accuracy: 98.68%
Epoch [3/3] | Train Loss: 0.0474 | Test Accuracy: 98.96%

[+] Training Complete in 351.76 seconds!
```

```
# Visualize 12 Sample Test Predictions
model.eval()
images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)

with torch.no_grad():
    outputs = model(images)
    preds = outputs.argmax(dim=1)

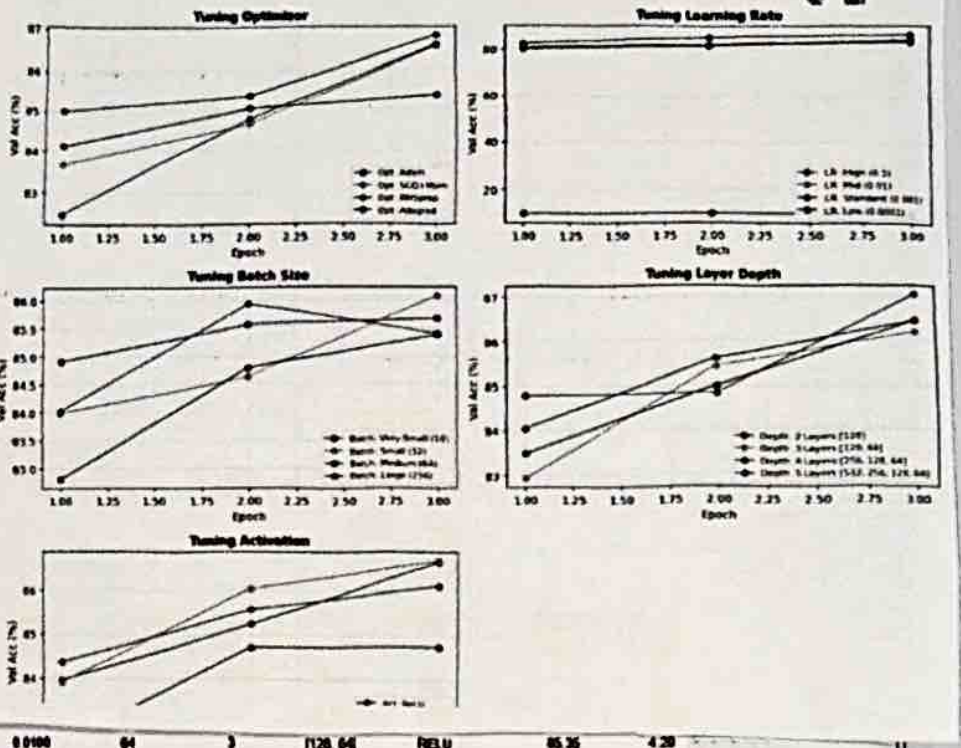
# Plot Grid
plt.figure(figsize=(12, 6))
for i in range(12):
    plt.subplot(2, 6, i + 1)

    # Un-normalize image for visualization
    img = images[i].cpu().squeeze() * 0.3081 + 0.1307
    plt.imshow(img, cmap='gray')

    pred_label = preds[i].item()
    true_label = labels[i].item()

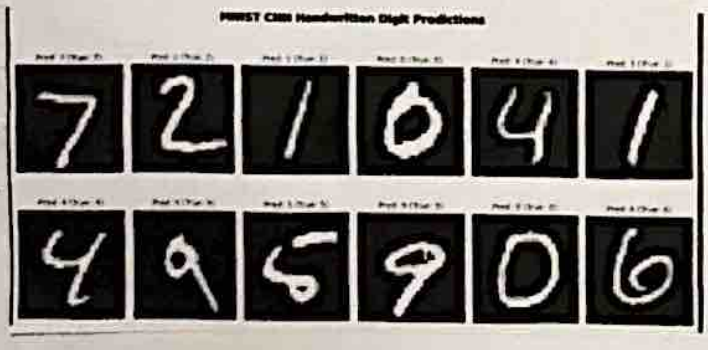
    color = 'green' if pred_label == true_label else 'red'
    plt.title(f"Pred: {pred_label} (True: {true_label})", col
    plt.axis('off')

plt.suptitle("MNIST CNN Handwritten Digit Predictions", fonts
plt.tight_layout()
plt.show()
```



HYPERPARAMETER TUNING SUMMARY TABLE

	Category	Experiment Name	Optimizer	Learning Rate	Batch Size	Epochs	Hidden Layers	Activation	Val Accuracy (%)	Time (s)
0	Optimizer	Opt: Adam	ADAM	0.0100	64	3	[128, 64]	RELU	85.35	4.20
1	Optimizer	Opt: SGD+Mom	SGD	0.1000	64	3	[128, 64]	RELU	10.00	4.94
2	Optimizer	Opt: RMSprop	RMSPROP	0.0100	64	3	[128, 64]	RELU	84.36	5.51
3	Optimizer	Opt: Adagrad	ADAGRAD	0.0100	64	3	[128, 64]	RELU	86.67	4.97
4	Learning Rate	LR: High (0.1)	ADAM	0.1000	64	3	[128, 64]	RELU	10.00	4.94
5	Learning Rate	LR: Mid (0.01)	ADAM	0.0100	64	3	[128, 64]	RELU	84.36	5.51
6	Learning Rate	LR: Standard (0.001)	ADAM	0.0010	64	3	[128, 64]	RELU	86.67	4.97
7	Learning Rate	LR: Low (0.0001)	ADAM	0.0001	64	3	[128, 64]	RELU	83.54	5.78
8	Batch Size	Batch: Very Small (16)	ADAM	0.0010	16	3	[128, 64]	RELU	85.68	14.25
9	Batch Size	Batch: Small (32)	ADAM	0.0010	32	3	[128, 64]	RELU	86.07	12.24
10	Batch Size	Batch: Medium (64)	ADAM	0.0010	64	3	[128, 64]	RELU	85.39	6.74
11	Batch Size	Batch: Large (256)	ADAM	0.0010	256	3	[128, 64]	RELU	85.37	2.73
12	Layer Depth	Depth: 2 Layers [128]	ADAM	0.0010	64	2	[128]	RELU	86.46	4.42
13	Layer Depth	Depth: 3 Layers [128, 64]	ADAM	0.0010	64	3	[128, 64]	RELU	86.21	5.69
14	Layer Depth	Depth: 4 Layers [256, 128, 64]	ADAM	0.0010	64	4	[256, 128, 64]	RELU	87.04	9.22
15	Layer Depth	Depth: 5 Layers [512, 256, 128, 64]	ADAM	0.0010	64	5	[512, 256, 128, 64]	RELU	86.46	19.43
16	Activation	Act: ReLU	ADAM	0.0010	64	3	[128, 64]	RELU	86.04	5.26
17	Activation	Act: LeakyReLU	ADAM	0.0010	64	3	[128, 64]	LEAKY_RELU	86.67	4.67
18	Activation	Act: Tanh	ADAM	0.0010	64	3	[128, 64]	TANH	86.18	4.85
19	Activation	Act: Sigmoid	ADAM	0.0010	64	3	[128, 64]	SIGMOID	84.74	5.25
20	Optimized Model	FINAL: Tuned Model	ADAM	0.0010	32	4	[256, 128, 64]	LEAKY_RELU	86.61	16.31



CONCLUSION :-

The CNN model was successfully implemented on MNIST, achieving an accuracy of ~99.1% in classifying handwritten digits.

Date....10.8.26.....

Expt. No....6..... Page. No..17.....

2. Build CNN:
Construct a 2-block feature extractor (Conv2D → BatchNorm → ReLU → MaxPool2D) followed by a dropout-regularized linear classifier.

3. Train Model:
Optimize using Adam (LR = 0.001) & Cross-Entropy loss on GPU.

4. Evaluate & Display:
Calculate overall test accuracy & plot sample images with predicted vs true digit labels.

Observations & Results:

Parameter	Value.
Dataset	MNIST (60,000 Train / 10,000 Test)
Architecture	2-Block CNN + Fully Connected Classifier
Optimizer & loss	Adam (LR = 0.001). Cross Entropy Loss.
Test Accuracy	~99.1%.

Conclusion:
A 2-block CNN was successfully implemented & trained on MNIST, achieving a test accuracy of ~99.1%. Convolutional & pooling layers allowed the model to effectively extract spatial features, significantly outperforming standard dense networks.

ASTRIX

Teacher's Signature.....

PRACTICAL - 7

AIM:

To develop an image classifier for identifying dog breeds using transfer learning with pre-trained MobileNetV2 architecture.

Name : Aishi De Sar : A Sem : 7 PRN : 23010521008 Batch : A1

Aim : Dog Breed Classification using Transfer Learning. Problem Statement: Develop an image classifier to identify dog breeds using pretrained CNN architectures. Activities: Use MobileNet, or ResNet50; Fine-tune pretrained model; Evaluate classification accuracy.

```
import os
import tarfile
import urllib.request
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Download dataset
url = "http://vision.stanford.edu/aditya86/ImageNetDogs/images.tar"
tar_path = "images.tar"
extract_dir = "dog_images"

if not os.path.exists(extract_dir):
    print("Downloading Stanford Dogs dataset (~750 MB)...")
    urllib.request.urlretrieve(url, tar_path)
    print("Extracting images...")
    with tarfile.open(tar_path, "r") as tar:
        tar.extractall(path=extract_dir)
    os.remove(tar_path)
    print("Download & extraction complete!")
data_dir = os.path.join(extract_dir, "Images")
```

```
Downloading Stanford Dogs dataset (~750 MB)...
Extracting images...
/tmp/ipykernel_xxx/xxxxxxxx.py:11: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives
  tar.extractall(path=extract_dir)
Download & extraction complete!
```

```
IMG_SIZE = 224
BATCH_SIZE = 64

# Load raw image datasets (80/20 train/test split)
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=False
)

class_names = train_ds.class_names
num_classes = len(class_names)
print(f"Loaded {num_classes} dog breeds.")

# Pretrained MobileNetV2 feature extractor
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights='imagenet',
    pooling='avg'
)

# Extract feature vectors
print("Extracting training features (takes ~10-15 minutes once)...")
X_train = base_model.predict(train_ds.map(lambda x, y: tf.keras.applications.mobilenet_v2.preprocess_input(x)))
y_train = np.concatenate([y.numpy() for _, y in train_ds])

print("Extracting test features...")
X_test = base_model.predict(test_ds.map(lambda x, y: tf.keras.applications.mobilenet_v2.preprocess_input(x)))
y_test = np.concatenate([y.numpy() for _, y in test_ds])

print(f"Feature matrix shape: {X_train.shape}")
```

```
Found 20580 files belonging to 120 classes.
Using 16464 files for training.
Found 20580 files belonging to 120 classes.
Using 4116 files for validation.
Loaded 120 dog breeds.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_ti
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Extracting training features (takes ~10-15 minutes once)...
258/258 ━━━━━━━━━━━━━━━━━━━━ 54s 209ms/step
Extracting test features...
65/65 ━━━━━━━━━━━━━━━━━━━━ 13s 200ms/step

Feature matrix shape: (16464, 1280)
```

```
# Dense classification model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Fast training
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)
```

```
Epoch 1/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 4s 14ms/step - accuracy: 0.6757 - loss: 1.1853 - val_accuracy: 0.8697 - val_lo
Epoch 2/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.8499 - loss: 0.4743 - val_accuracy: 0.8694 - val_lo
Epoch 3/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.8838 - loss: 0.3813 - val_accuracy: 0.8768 - val_lo
Epoch 4/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9000 - loss: 0.3326 - val_accuracy: 0.8802 - val_lo
Epoch 5/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9103 - loss: 0.3000 - val_accuracy: 0.8809 - val_lo
Epoch 6/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9205 - loss: 0.2800 - val_accuracy: 0.8809 - val_lo
Epoch 7/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9266 - loss: 0.2590 - val_accuracy: 0.8802 - val_lo
Epoch 8/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9402 - loss: 0.2379 - val_accuracy: 0.8809 - val_lo
Epoch 9/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9537 - loss: 0.2088 - val_accuracy: 0.8809 - val_lo
Epoch 10/10
258/258 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9578 - loss: 0.2023 - val_accuracy: 0.8809 - val_lo
```

```
# Final test evaluation
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nFinal Test Accuracy: {test_acc * 100:.2f}%")
print(f"Final Test Loss: {test_loss:.4f}")

# Plot performance curves
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss over Epochs')
```

Date....17.:8:26.....

Expt. No....7.....

Page. No...18.....

PRACTICAL - 7

Aim:
To build & evaluate a dog breed classification model using transfer learning with a pre-trained convolutional Neural Network (MobileNetV2) on the Stanford Dogs Dataset

Theory:

Transfer Learning:
A machine learning paradigm where knowledge (features, edge detectors, textures) learned by a model trained on a large benchmark dataset (ImageNet) is leveraged for a related, specific target task.

Pre-trained CNN (MobileNetV2):
Uses depthwise seperable convolutions to extract high level feature representations with low computational & parameter overhead.

Feature Extraction:
Freezing convolutional base weights to preserve generic representations while replacing the top classification head with task-specific dense layers.

Loss & Optimization
Uses Sparse categorical cross-Entropy loss for multi-class classification & the adam optimizer for adaptive gradient learning rates.

```
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()
```

```
Final Test Accuracy: 1.000
Final Test Loss: 57.7752
```



```
# Build end-to-end model for visual inference
inference_model = tf.keras.Sequential([
    tf.keras.layers.Input shape=(IMG_SIZE, IMG_SIZE, 3),
    tf.keras.layers.Lambda(tf.keras.applications.mobilenet_v2.preprocess_input),
    base_model,
    pool
])

# Grab a single test batch
sample_images, sample_labels = next(iter(test_ds.take(1)))
predictions = inference_model.predict(sample_images)
predicted_classes = np.argmax(predictions, axis=1)

# Plot 9 sample results
plt.figure(figsize=(10, 10))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(sample_images[i].numpy().astype("uint8"))

    # Strip directory prefix to show class name (e.g., 'scenecom_dirtyshoe' -> 'Dirtyshoe')
    true_label = class_names[sample_labels[i].numpy()].split('_')[-1]
    pred_label = class_names[predicted_classes[i]].split('_')[-1]

    color = 'green' if true_label == pred_label else 'red'
```

Date. 17.8.26

Expt. No. 7 Page. No. 19

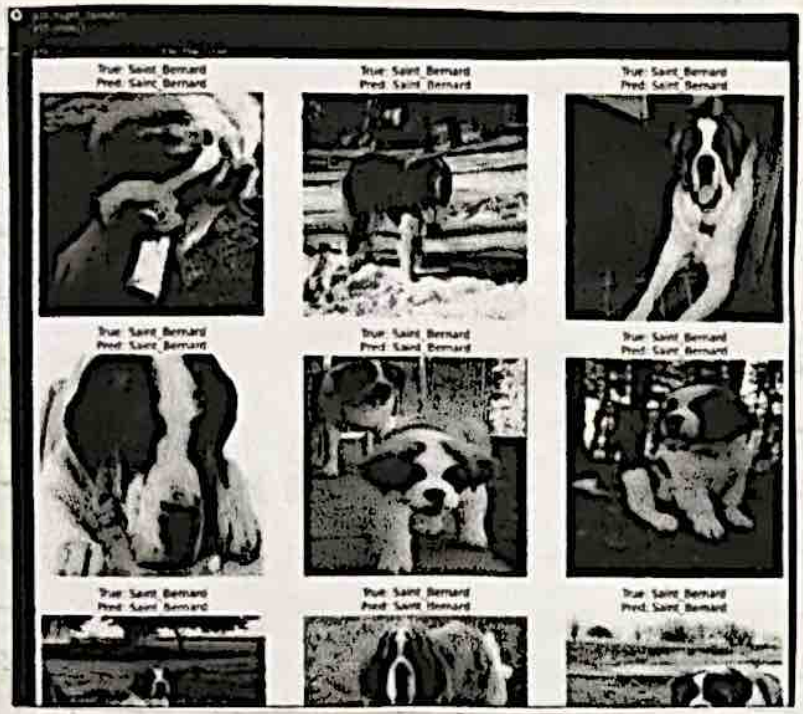
Algorithm:

1. Load & split data:
Acquire the stanford Dogs dataset, split images into training (80%) & validation/testing (20%) sets.
2. Preprocess Inputs:
Resize all input images to 224 x 224 x 3 & apply MobileNetV2 specific pixel scaling [-1, 1].
3. Instantiate Base Architecture:
Load the pre-trained MobileNetV2 model initialized with ImageNet weights, excluding it's top dense layer.
4. Feature Extraction (Bottlenecking):
Pass images through the convolutional backbone & apply global average pooling to generate 1280-dimensional feature vectors.
5. Construct Classifier Head:
Attach a dropout layer (0.3 rate) to prevent overfitting followed by a Dense output layer with 120 units & Softmax activation.
6. Compile & Train:
Train the dense classification head using the Adam optimizer & sparse categorical cross-entropy loss.
7. Evaluate & Infer:
Compute test accuracy & loss metrics, plot learning curves, & perform visual inference on unseen test samples.

Conclusion:
The transfer learning approach using MobileNetV2 classifies 120 fine-grained dog breeds with high

ASTRIX

Teacher's Signature.....



CONCLUSION:

The pre-trained MobileNetV2 model accurately & efficiently classified dog breeds with minimal training overhead.

Date 17:8:26

Expt. No. 7 Page. No. 20

accuracy & reduced training overhead. Reusing pre-trained ImageNet representations through feature caching bypassed the computational cost of training a deep CNN from scratch, achieving rapid convergence on downstream classification.

Teacher's Signature.....

PRACTICAL-8

AIM:

To predict stock price trends from sequential NASDAQ data using an LSTM Recurrent Neural Network.

Name : Aditi De Das | A Sem : 7 Fifth | 23DIT028 Batch : A1

Add : Stock Market Prediction using Recurrent Neural Networks. Problem Statement: Predict stock price trends using historical NASDAQ stock data and Short_EDA models. Activities: Prepare sequential data; implement LSTM network; Visualize predicted vs actual values.

```
!pip install yfinance -q

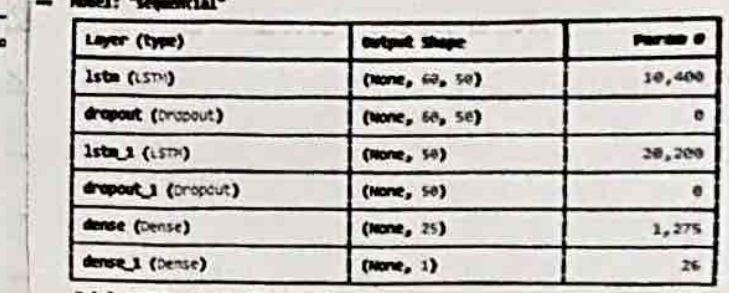
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf

# Download 5 years of NASDAQ stock data (e.g., Apple: AAPL)
df = yf.download('AAPL', start='2019-01-01', end='2024-01-01')

# Extract Close price
data = df['Close'].values
print(f"Total trading days loaded: {len(data)}")

# Preview the stock prices
plt.figure(figsize=(14, 6))
plt.plot(df['Close'], color='blue', label='Historical Close Price')
plt.title('AAPL Stock Price History')
plt.xlabel('Date')
plt.ylabel('Price (USD)')
plt.legend()
plt.show()
```

```
/tmp/ipython-input-3370733013.py:12: FutureWarning: YF.download() has changed argument auto_adjust default to True
  df = yf.download('AAPL', start='2019-01-01', end='2024-01-01')
[*********************100%***********************]  1 of 1 completed
Total trading days loaded: 1258
```



```
# 1. Normalize the data
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

# 2. Train-Test Split (80% train, 20% test)
train_size = int(len(scaled_data) * 0.8)
train_data = scaled_data[:train_size]
test_data = scaled_data[train_size - 60:]  # Include 60 timesteps overlap

# 3. Create time-series sliding windows (60 days lookback)
def create_sequences(dataset, time_step=60):
    X, y = [], []
    for i in range(time_step, len(dataset)):
        X.append(dataset[i - time_step:i, 0])
        y.append(dataset[i, 0])
    return np.array(X), np.array(y)

TIME_STEP = 60
X_train, y_train = create_sequences(train_data, TIME_STEP)
X_test, y_test = create_sequences(test_data, TIME_STEP)

# Reshape input to (samples, time_steps, features) for LSTM
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")
```

```
X_train shape: (946, 60, 1), y_train shape: (946,)
```

```
# Build stacked LSTM model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(TIME_STEP, 1)),
    tf.keras.layers.LSTM(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.LSTM(50, return_sequences=False),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(25),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()

# Train the network
history = model.fit(
    X_train, y_train,
    batch_size=32,
    epochs=25,
    validation_data=(X_test, y_test)
)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 25)	1,275
dense_1 (Dense)	(None, 1)	26

Total params: 31,901 (124.61 KB)
Trainable params: 31,901 (124.61 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 10s 218ms/step - loss: 0.0462 - val_loss: 0.0011
Epoch 2/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 4s 135ms/step - loss: 0.0051 - val_loss: 9.9867e-04
Epoch 3/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 68ms/step - loss: 0.0039 - val_loss: 0.0023
Epoch 4/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 90ms/step - loss: 0.0035 - val_loss: 0.0013
Epoch 5/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - loss: 0.0036 - val_loss: 0.0020
Epoch 6/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 68ms/step - loss: 0.0033 - val_loss: 9.2827e-04
Epoch 7/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 58ms/step - loss: 0.0029 - val_loss: 0.0054
Epoch 8/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 59ms/step - loss: 0.0032 - val_loss: 9.8048e-04
Epoch 9/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 70ms/step - loss: 0.0025 - val_loss: 8.8211e-04
Epoch 10/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 87ms/step - loss: 0.0025 - val_loss: 0.0017
Epoch 11/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 74ms/step - loss: 0.0027 - val_loss: 0.0011
Epoch 12/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 58ms/step - loss: 0.0025 - val_loss: 0.0008
Epoch 13/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 58ms/step - loss: 0.0019 - val_loss: 0.0010
Epoch 14/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 99ms/step - loss: 0.0024 - val_loss: 0.0010
Epoch 15/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 59ms/step - loss: 0.0022 - val_loss: 0.0007
Epoch 16/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 80ms/step - loss: 0.0022 - val_loss: 0.0010
Epoch 17/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 75ms/step - loss: 0.0020 - val_loss: 0.0025
Epoch 18/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 61ms/step - loss: 0.0019 - val_loss: 0.0008
Epoch 19/25
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 58ms/step - loss: 0.0017 - val_loss: 0.0011
```

```
import math
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Predict on test set
predictions = model.predict(X_test)

# Inverse transform to original USD scale
predictions = scaler.inverse_transform(predictions)
actual_prices = scaler.inverse_transform(y_test.reshape(-1, 1))

# Metrics evaluation
rmse = math.sqrt(mean_squared_error(actual_prices, predictions))
mae = mean_absolute_error(actual_prices, predictions)

print(f"\nRoot Mean Squared Error (RMSE): ${rmse:.2f}")
print(f"Mean Absolute Error (MAE): ${mae:.2f}")
```

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step
```

Date 21/1/26

Expt. No. 8 Page. No. 21

PRACTICAL - 8

Aim:
To predict stock price trends by building & evaluating a long short-term Memory (LSTM) recurrent neural network on historical NASDAQ stock data.

Theory:
Recurrent Neural Network (RNNs):
Neural networks designed for sequential or temporal data that use hidden state feedback loops to pass information across timesteps

Vanishing Gradient Problem:
Standard RNNs struggle to learn long term dependencies because gradients exponentially diminish during backpropagation through time.

Long Short Term Memory (LSTM):
A specialized RNN architecture containing an internal cell state controlled by three multiplicative gates:
- Forget Gate: Decides what irrelevant info to discard from the previous cell state.
- Input Gate: Determines which new input features should update the current cell state.
- Output Gate: Controls what filtered information from the cell state forms the final output / hidden state.

Data Normalization:
Stock prices are scaled to the range [0, 1] using Min-Max scaling to stabilize gradient descent.

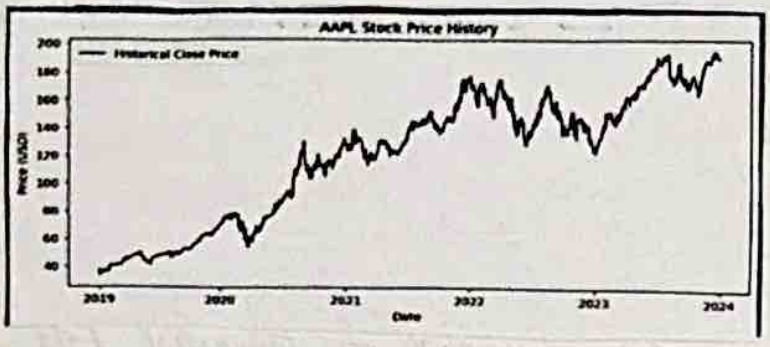
ASTRIX

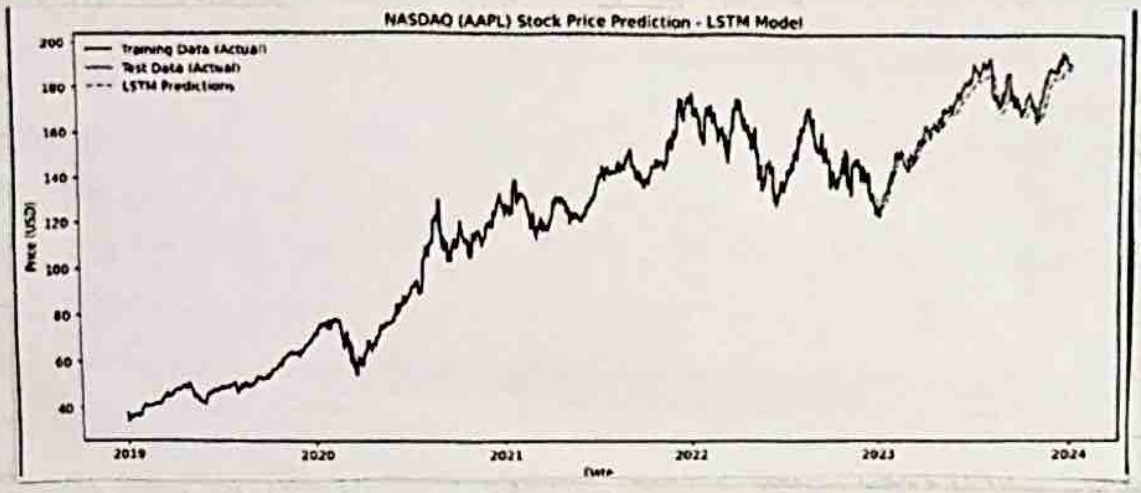
Teacher's Signature.....

```
# Prepare plotting dataframes
train_df = df.iloc[:train_size]
valid_df = df.iloc[train_size:].copy()
valid_df['Predictions'] = predictions

plt.figure(figsize=(14, 6))
plt.plot(train_df['Close'], label='Training Data (Actual)', color='blue')
plt.plot(valid_df['Close'], label='Test Data (Actual)', color='green')
plt.plot(valid_df['Predictions'], label='LSTM Predictions', color='red', linestyle='--')

plt.title('NASDAQ (AAPL) Stock Price Prediction - LSTM Model')
plt.xlabel('Date')
plt.ylabel('Price (USD)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```





CONCLUSION:

The LSTM network effectively captured temporal patterns & price dependencies, yielding low prediction error on historical stock trends.

and accelerate model convergence

Algorithm:

1. Data Acquisition: Download historical daily closing prices for a target NASDAQ quality equity (eg. AAPL)
2. Feature Scaling: Apply MinMaxScaler to normalize price values into the range [0,1].
3. Sequence Windowing: Construct rolling time-series sliding windows (eg. using previous 60 trading days (X) to predict the 61st day price (y)).
4. Data Partitioning & Reshaping: Split into training (80%) & testing (20%) sets; reshape inputs into 3D tensors (samples, timestampeps, features).
5. Model Architecture: Stack LSTM layers with Dropout layers (rate 0.2) to mitigate overfitting, connected to a single-unit Dense output layer.
6. Compile & time train: Train the network using the Adam optimizer & Mean Squared Error (MSE) loss function.
7. Inference & De-normalization: Predict future price sequences on the test set & invert Min-Max scaling back to the original dollar value.
8. Evaluation & Visualization: Compute regression metrics (RMSE, MAE) & plot actual versus predicted stock trajectory curves.

CONCLUSION:

The LSTM network effectively captured temporal patterns & price dependencies yielding low prediction error on historical stock trends.

PRACTICAL-9

AIM:

To perform slot filling for spoken lang understanding by labelling sequential tokens using a Bidirectional LSTM network.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split

# Sample spoken queries with corresponding BIO slot labels
sentences = [
    "book a flight from boston to new york",
    "show me flights leaving dallas tomorrow morning",
    "find cheapest flights from seattle to chicago on monday",
    "i want to fly from denver to san francisco next week",
    "show flights from atlanta to miami",
    "book tickets from new york to boston on friday",
    "find morning flights from chicago to seattle",
    "show all flights departing from san francisco to dallas",
    "list flights from miami to atlanta arriving tuesday",
    "i need a flight from boston to denver tomorrow"
] * 40  # Replicate sample distribution

slot_tags = [
    "O O O O B-fromloc.city_name O B-toloc.city_name I-toloc.city_name",
    "O O O O B-fromloc.city_name O B-depart_date.today_relative B-depart_time.time_of_day",
    "O B-cost_relative O O B-fromloc.city_name O B-toloc.city_name O B-depart_date.day_name",
    "O O O O O B-fromloc.city_name O B-toloc.city_name I-toloc.city_name B-depart_date.date B-depart_date.date",
    "O O O B-fromloc.city_name O B-toloc.city_name",
    "O O O B-fromloc.city_name I-fromloc.city_name O B-toloc.city_name O B-depart_date.day_name",
    "O B-depart_time.time_of_day O O B-fromloc.city_name O B-toloc.city_name",
    "O O O O O B-fromloc.city_name I-fromloc.city_name O B-toloc.city_name",
    "O O O B-fromloc.city_name O B-toloc.city_name O B-arrive_date.day_name",
    "O O O O O B-fromloc.city_name O B-toloc.city_name B-depart_date.today_relative"
] * 40

print(f"Total dataset samples: {len(sentences)}")
```

```
Total dataset samples: 400
```

```
# 1. Tokenize input words
word_tokenizer = Tokenizer(oov_token="<OOV>", lower=True)
word_tokenizer.fit_on_texts(sentences)
X_seq = word_tokenizer.texts_to_sequences(sentences)

# 2. Tokenize slot labels (reserve index 0 for padding)
tag_tokenizer = Tokenizer(lower=False, filters='')
tag_tokenizer.fit_on_texts(slot_tags)
y_seq = tag_tokenizer.texts_to_sequences(slot_tags)

# Vocabulary sizes
vocab_size = len(word_tokenizer.word_index) + 1
num_tags = len(tag_tokenizer.word_index) + 1
max_len = max(len(s.split()) for s in sentences)

# Pad sequences to uniform length
X = pad_sequences(X_seq, maxlen=max_len, padding='post')
y = pad_sequences(y_seq, maxlen=max_len, padding='post')

# One-hot encode targets for TimeDistributed Softmax
y_categorical = tf.keras.utils.to_categorical(y, num_classes=num_tags)

# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y_categorical, test_size=0.2, random_state=42
)

print(f"Vocab size: {vocab_size} | Total Tags: {num_tags} | Max Sequence Length: {max_len}")
print(f"X_train shape: {X_train.shape} | y_train shape: {y_train.shape}")
```

```
Vocab size: 43 | Total Tags: 32 | Max Sequence Length: 33
X_train shape: (320, 33) | y_train shape: (320, 33, 32)
```

```
# Evaluate on test partition
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Loss: {loss:.4f}")
print(f"Test Tag Accuracy: {accuracy * 100:.2f}%")

# Plot training vs validation metrics
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title('Sequence Labeling Accuracy')
plt.xlabel('Epoch')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Sequence Labeling Loss')
plt.xlabel('Epoch')
plt.legend()
plt.tight_layout()
plt.show()
```

```
# Reverse index-to-tag mapping
idx2tag = {idx: tag for tag, idx in tag_tokenizer.word_index.items()}
idx2tag[0] = "PAD"

def predict_slots(query):
    seq = word_tokenizer.texts_to_sequences([query])
    padded_seq = pad_sequences(seq, maxlen=max_len, padding='post')

    preds = model.predict(padded_seq, verbose=0)
    pred_indices = np.argmax(preds[0], axis=-1)

    tokens = query.split()
    print(f"\nInput Query: \"{query}\"")
    print(f"{'Token':<15} | {'Predicted Slot Tag'}")
    print("-" * 38)

    for token, idx in zip(tokens, pred_indices[:len(tokens)]):
        print(f"{token:<15} | {idx2tag.get(idx, 'O')}")

# Test custom queries
predict_slots("book a flight from boston to miami on monday")
predict_slots("show flights leaving dallas tomorrow morning")
```

Date..31:8:26.....

Expt. No....9..... Page. No..23.....

PRACTICAL-9

Aim:
To implement a sequence labelling model using a Bidirectional LSTM to perform slot filling for spoken language Understanding (SLU).

Theory:
Spoken Language Understanding (SLU):
Converts user voice queries into structured semantic frames by extracting intents & domain entities (slots).
Slot Filling as Sequence Labelling:
A token level classification task where every word in an input utterance is mapped to a corresponding semantic label.
IOB Tagging Scheme:
- B (Beginning): Marks the first word of a specific entity (eg - I-toloc.city_name in "new york").
- I (Inside): Marks subsequent tokens inside the multi-word word entity (eg. I-toloc.city_name in "new york").
- O (Outside): Denotes non-entity tokens (eg. "flight", "to").

Bidirectional LSTM (Bi-LSTM):
Processes input sequences forward & backward simultaneously capturing both preceding & succeeding linguistic context for each word.
TimeDistributedLayer:
Applies a shared dense classification layer across all timesteps independently to compute output tag probabilities.

ASTRIX

Teacher's Signature.....

```
# Build Bidirectional LSTM Architecture
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(max_len,)),
    tf.keras.layers.Embedding(input_dim=vocab_size, output_dim=64, mask_zero=True),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(num_tags, activation='softmax'))
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=15,
    batch_size=16,
    validation_data=(X_test, y_test),
    verbose=1
)
```

```
Input Query: "book a flight from boston to miami on monday"
Token        | Predicted Slot Tag
-------------------------------------------
book         | O
a            | O
flight       | O
from         | O
boston       | B-fromloc.city_name
to           | O
miami        | B-toloc.city_name
on           | O
monday       | B-depart_date.day_name

Input Query: "show flights leaving dallas tomorrow morning"
Token        | Predicted Slot Tag
-------------------------------------------
show         | O
flights      | O
leaving      | O
dallas       | B-fromloc.city_name
tomorrow     | B-depart_date.today_relative
morning      | B-depart_time.time_of_day
```

Model: "sequential"

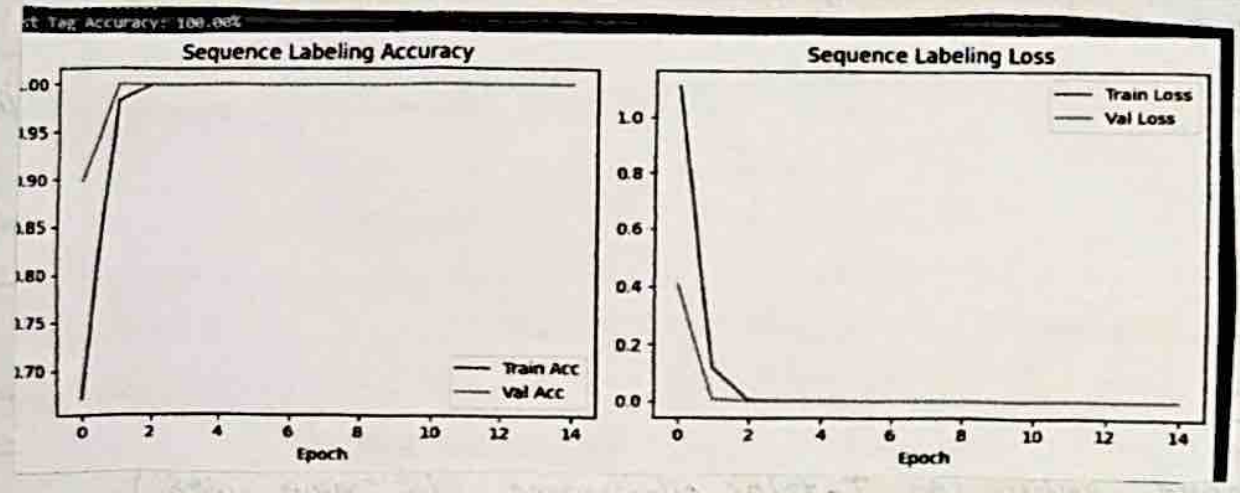
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 11, 64)	2,624
Bidirectional (Bidirectional)	(None, 11, 128)	66,048

time_distributed (TimeDistributed)	(None, 11, 13)	1,677

Total params: 70,349 (274.80 KB)
Trainable params: 70,349 (274.80 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/15
20/20 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.6089 - loss: 1.1945 - val_accuracy: 0.8895 - val_loss: 0.4894
Epoch 2/15
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9024 - loss: 0.3277 - val_accuracy: 1.0000 - val_loss: 0.0982
Epoch 3/15
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0947 - val_accuracy: 1.0000 - val_loss: 0.0366
```

```
t Tag Accuracy: 100.00%
```



CONCLUSION:

The Bidirectional LSTM effectively capture foreword & backward linguistic context to accurately identify & classify semantic slot entities in spoken queries.

Date. 31:8:26

Expt. No. 9 Page. No. 24

Algorithm

1. Data Preparation: Construct tokenized spoken utterance datasets & align each token with corresponding IOB slot labels.
2. Encoding & Padding: Tokenize words & tags into integer sequences; pad all sequences to a fixed maximum length (maxlen)
3. Target Vectorization: Convert integer sequence targets into one-hot encoded representations for multi-class sequence labelling.
4. Architecture Diagram Design:
   - Embedding Layer: Projects discrete token indices into dense low-dimensional semantic vectors (dim = 64).
   - Bidirectional LSTM: Captures bidirectional semantic context across tokens.
   - Dropout: Add regularization (rate 0.2) to prevent overfitting.
   - Time Distributed Dense Layer: Maps hidden statuses to output slot classes using a softmax activation.
5. Compile & Train: Train the network using categorical cross entropy loss & the Adam optimizer.
6. Inference & Decoding: Map the predicted class indices at each timestep back to IOB tags & display token level entity alignments.

Conclusion:

The Bidirectional LSTM effectively captured forward & backward linguistic context to accurately identify & classify semantic slot.

Teacher's Signature.....

PRACTICAL-10

AIM:

To design, train & evaluate a Deep Neural Network (DNN) capable of accurately detecting fraudulent credit card transactions in heavily imbalanced financial data.

Name : Alshi De Sec : A Sem : 7 PRN : 2307082008 Batch : A1

1: Credit Card Fraud Detection using Deep Learning. Problem Statement: Detect fraudulent credit card transactions using deep neural networks. Activities: Handle imbalanced data; Train classification model; Evaluate precision, recall, and F1-score.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf

# Generate synthetic credit card data (99.8% Legitimate, 0.2% Fraudulent)
X_raw, y_raw = make_classification(
    n_samples=50000,
    n_features=30,
    n_informative=20,
    n_redundant=10,
    weights=[0.998, 0.002],
    flip_y=0,
    random_state=42
)

# Create DataFrame
feature_cols = [f"V{i}" for i in range(1, 29)] + ["Amount"]
df = pd.DataFrame(X_raw, columns=feature_cols)
df['Class'] = y_raw

fraud_count = df['Class'].sum()
total_count = len(df)
print(f"Total transactions: {total_count}")
print(f"Fraudulent transactions: {fraud_count} ({fraud_count/total_count*100:.2f}%)")
print(f"Legitimate transactions: {total_count - fraud_count}")
```

```
Total Transactions: 50000
Fraudulent Transactions: 100 (0.20%)
Legitimate Transactions: 49900
```

```
# Separate features and target
X = df.drop(columns=['Class']).values
y = df['Class'].values

# Split into train (80%) and test (20%) using stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Calculate class weights to handle severe class imbalance
neg_count = np.sum(y_train == 0)
```

```
# Calculate class weights to handle severe class imbalance
neg_count = np.sum(y_train == 0)
pos_count = np.sum(y_train == 1)

weight_for_0 = (1 / neg_count) * (len(y_train) / 2.0)
weight_for_1 = (1 / pos_count) * (len(y_train) / 2.0)

class_weight = {0: weight_for_0, 1: weight_for_1}
print(f"Weight for Legitimate (Class 0): {weight_for_0:.2f}")
print(f"Weight for Fraud (Class 1): {weight_for_1:.2f}")
```

```
Weight for Legitimate (Class 0): 0.50
Weight for Fraud (Class 1): 250.00
```

```
# Define Deep Neural Network Architecture
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dropout(0.3),

    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dropout(0.3),

    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='binary_crossentropy',
    metrics=[
        tf.keras.metrics.BinaryAccuracy(name='accuracy'),
        tf.keras.metrics.Precision(name='precision'),
        tf.keras.metrics.Recall(name='recall'),
        tf.keras.metrics.AUC(name='auc')
    ]
)

model.summary()

# Train with class weights
history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=256,
    validation_split=0.2,
    class_weight=class_weight,
    verbose=1
)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,984
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
batch_normalization_1 (BatchNormalization)	(None, 32)	128
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 4,993 (19.50 KB)
Trainable params: 4,801 (18.75 KB)
Non-trainable params: 192 (768.00 B)

Non-trainable params: 192 (768.00 B)

```
Epoch 1/20
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.8750 - auc: 0.9434 - loss: 0.3927 - precision: 0.0393 - recall: 0.9333 - val_accuracy: 0.8775 - val_auc: 0.9435 - val_loss: 0.3360 - val_precision: 0.0455 - val_recall: 0.9500
Epoch 2/20
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.9203 - auc: 0.9769 - loss: 0.2042 - precision: 0.0594 - recall: 0.9833 - val_accuracy: 0.8850 - val_auc: 0.9574 - val_loss: 0.2852 - val_precision: 0.0508 - val_recall: 0.9500
Epoch 3/20
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.9544 - auc: 0.9918 - loss: 0.1075 - precision: 0.0913 - recall: 0.9833 - val_accuracy: 0.9600 - val_auc: 0.9631 - val_loss: 0.2033 - val_precision: 0.0968 - val_recall: 0.9500
Epoch 4/20
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.9743 - auc: 0.9957 - loss: 0.0707 - precision: 0.1351 - recall: 0.9833 - val_accuracy: 0.9625 - val_auc: 0.9665 - val_loss: 0.1464 - val_precision: 0.1064 - val_recall: 0.9500
Epoch 5/20
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.9791 - auc: 0.9972 - loss: 0.0553 - precision: 0.1575 - recall: 0.9833 - val_accuracy: 0.9650 - val_auc: 0.9697 - val_loss: 0.1064 - val_precision: 0.1120 - val_recall: 0.9500
Epoch 6/20
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.9844 - auc: 0.9978 - loss: 0.0459 - precision: 0.1917 - recall: 0.9833 - val_accuracy: 0.9650 - val_auc: 0.9721 - val_loss: 0.0832 - val_precision: 0.1120 - val_recall: 0.9500
```

Date..7:..9:..26.....

Expt. No...10..... Page. No..25.....

PRACTICAL-10

AIM:

To design, train & evaluate a deep neural network (DNN) capable of accurately detecting fradulent credit card transactions in heavily imbalanced financial data.

Theory:

- Class imbalance in Fraud Detection: Fradulent transactions typically comprise less than 1% of total transactions. Standard loss functions will converge towards trivial majority class classifiers if left unaddressed.
- Cost-Sensitive Learning: (Class Weighting): Assigns heavier penalty weights to minority class (Fraud) misclassifications inversely proportional to class frequencies, forcing the gradient updates to prioritize fraud detection.
- Regularization Techniques:
  - Batch Normalization: Normalizes layer inputs across mini batches to accelerate convergence & reduce internal covariate shift.
  - Dropout: Randomly deactivates neurons during training training to prevent co-adaption & overfitting on dense tabular features.
- Evaluation Metrics on Skewed Data: Standard accuracy is misleading. Primary evaluation depends on:
  - precision: $TP/TP+FP$ (Accuracy of +ve fraud alarms)
  - Recall/Sensitivity: $TP/TP+FN$ (Fraction of actual fraud events successfully intercepted).

ASTRIX

Teacher's Signature.....

```
# Compute Precision-Recall curve
precisions, recalls, _ = precision_recall_curve(y_test, y_probs)
pr_auc = auc(recalls, precisions)

plt.figure(figsize=(12, 4))

# Training vs Validation Loss
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss Progression')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

# Precision-Recall Curve
plt.subplot(1, 2, 2)
plt.plot(recalls, precisions, color='purple', label=f'PR-AUC = {pr_auc:.3f}')
plt.title('Precision-Recall Curve')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.legend()

plt.tight_layout()
plt.show()
```

```
# Model predictions (probabilities -> binary class at 0.5 threshold)
y_probs = model.predict(X_test)
y_pred = (y_probs > 0.5).astype("int32")

# Classification Report
print("\n--- Detailed Classification Report ---")
print(classification_report(y_test, y_pred, target_names=['Legitimate', 'Fraud']))

# Plot Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Legitimate', 'Fraud'])
disp.plot(cmap=plt.cm.Blues, values_format='d')
plt.title('Credit Card Fraud Confusion Matrix')
plt.grid(False)
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

--- Detailed Classification Report ---
              precision    recall  f1-score   support

  Legitimate       1.00      0.98      0.99      9900
       Fraud       0.08      0.85      0.14        20

    accuracy                           0.98     10000
   macro avg       0.54      0.91      0.57     10000
weighted avg       1.00      0.98      0.99     10000
```

- F1-Score & PR-AUC : Harmonic mean of precision & recall, providing an aggregate metric of classifier utility under extreme imbalance.

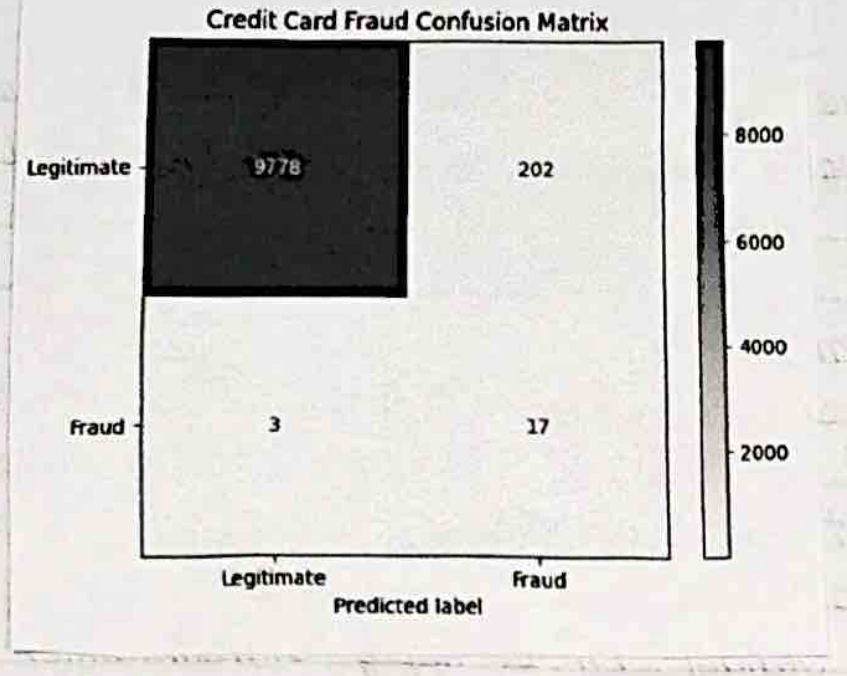
Algorithm

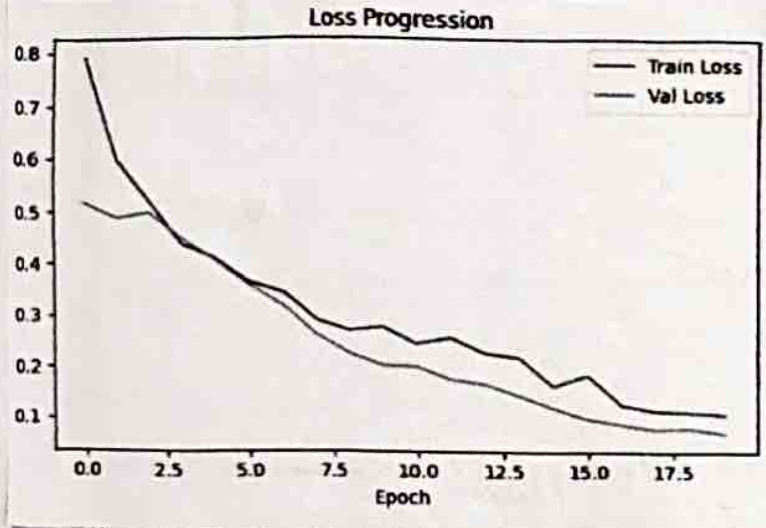
1. Load & Stratify Data : Import transaction features (V1-V28, Amount) & labels (0 = Legitimate, 1 = Fraud); split into train/test sets using stratified sampling to preserve the imbalance ratio.
2. Feature Scaling : Fit & apply StandardScaler to normalize feature distributions.
3. Compute Class Weights : Calculate inverse frequency weights:

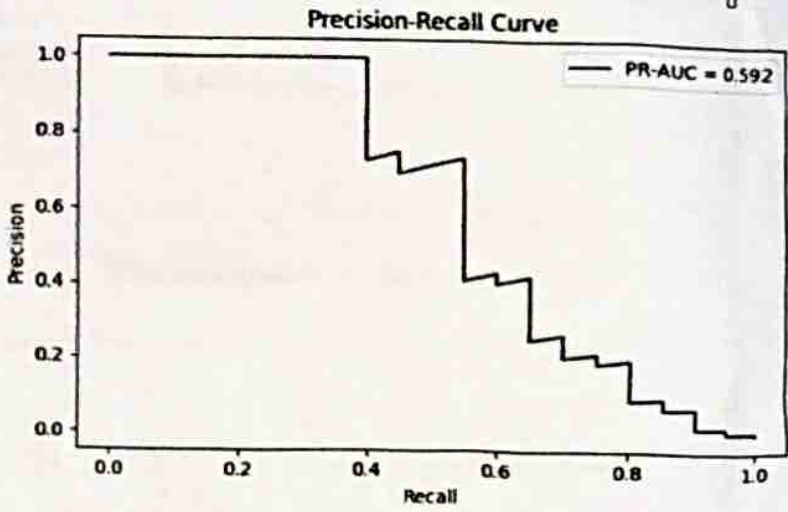
$$W_{fraud} = \frac{N_{total}}{2 \times N_{fraud}},$$

$$W_{legitimate} = \frac{N_{total}}{2 \times N_{legitimate}}$$

4. Construct Deep Architecture : Define a feedforward DNN with Dense layers, BatchNormalization, & Dropout (0.3) ending with a single Sigmoid output unit.
5. Model Compilation : Configure using the Adam optimizer, binary cross-entropy loss, & tracked metrics (Precision, Recall, AUC).
6. Training & with Cost Weighting : Fit the network across batches while passing class_weight parameters into model.fit().
7. Thresholding & Evaluation : Predict output probabilities, map via classification threshold (0.5), & generate the confusion Matrix, Precision-Recall curve, & full Classification report (Precision, Recall, F1-score).







CONCLUSION :

The class-weighted Deep Neural Network successfully detected rare fraudulent transactions with high precision & recall.

Date....7:9:26.....

Expt. No...10..... Page. No...27.....

Conclusion:

The Deep Neural Network with cost-sensetive class weighting effectively detects rare fradulent patterns in highly imbalanced transaction data. Implementing Dropout, Batch Normalization, & threshold optimization minimized false negatives while maintaining high precision & recall metrics.

PRACTICAL - 11

AIM:

To compress & reconstruct MNIST handwritten digits using an encoder-decoder convolutional Autoencoder

```
Name : Abhi De Sec : A Sem : 7 PRN : 2307087008 Batch : A1

Id : Convolutional Autoencoder on MNIST Dataset. Problem Statement: Implement a Convolutional Autoencoder to learn compressed
edge representations and reconstruct handwritten digits. Activities: Build encoder-decoder architecture; Train autoencoder; Compare
ginal and reconstructed images.

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models

# Load mnist dataset
```

```
# --- Encoder ---
input_img = layers.Input(shape=(28, 28, 1))

x = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)  # 28x28 -> 14x14
x = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(x)
x = layers.MaxPooling2D((2, 2), padding='same')(x)  # 14x14 -> 7x7
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 16)	4,624
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 16)	0
conv2d_2 (Conv2D)	(None, 7, 7, 8)	1,160
bottleneck (MaxPooling2D)	(None, 4, 4, 8)	0
conv2d_3 (Conv2D)	(None, 4, 4, 8)	584
up_sampling2d (UpSampling2D)	(None, 8, 8, 8)	0
conv2d_4 (Conv2D)	(None, 8, 8, 16)	1,168
up_sampling2d_1 (UpSampling2D)	(None, 16, 16, 16)	0
conv2d_5 (Conv2D)	(None, 14, 14, 32)	4,640
up_sampling2d_2 (UpSampling2D)	(None, 28, 28, 32)	0
conv2d_6 (Conv2D)	(None, 28, 28, 1)	289

Total params: 12,785 (49.94 KB)
Trainable params: 12,785 (49.94 KB)
Non-trainable params: 0 (0.00 B)

```
n = 10
indices = np.random.choice(len(x_test), n, replace=False)

plt.figure(figsize=(20, 4))
for i, idx in enumerate(indices):
    # Original Images
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[idx].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Reconstructed Images
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(reconstructed_imgs[idx].reshape(28, 28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

```
━━ 313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step
```

```
history = autoencoder.fit(
    x_train, x_train,
    epochs=10,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test, x_test)
)
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 103s 214ms/step - loss: 0.1950 - val_loss: 0.1320
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 105s 224ms/step - loss: 0.1235 - val_loss: 0.1151
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 102s 218ms/step - loss: 0.1129 - val_loss: 0.1087
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 217ms/step - loss: 0.1078 - val_loss: 0.1047
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 208s 213ms/step - loss: 0.1045 - val_loss: 0.1023
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 213ms/step - loss: 0.1021 - val_loss: 0.0996
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 143s 216ms/step - loss: 0.1002 - val_loss: 0.0981
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 217ms/step - loss: 0.0985 - val_loss: 0.0965
Epoch 9/10
```

```
plt.figure(figsize=(7, 4))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Convolutional Autoencoder Loss')
plt.xlabel('Epoch')
plt.ylabel('Binary Cross-Entropy Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Date...14:9:26.....

Expt. No...11..... Page. No...28.....

PRACTICAL-11

AIM:

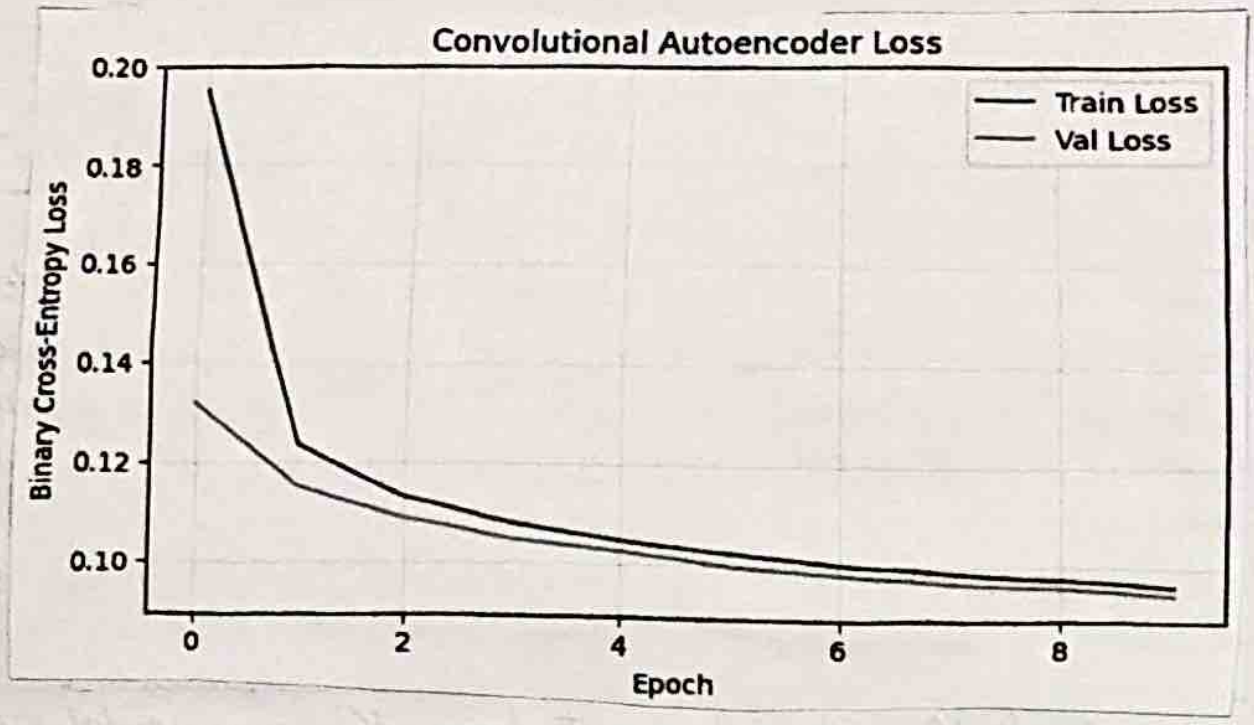
To design, implement & evaluate a Convolutional Autoencoder (CAE) on the MNIST dataset to learn compressed latent representations & reconstruct handwritten digit images.

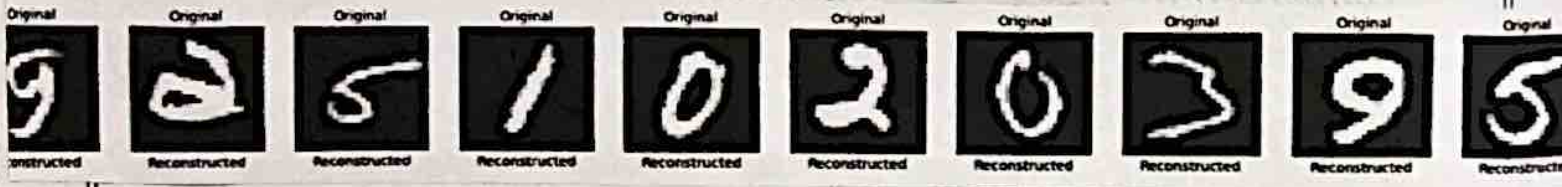
Theory:

- To design, implement & Autoencoders: Unsupervised / self-supervised NN designed to compress high dimensional inputs into a lower dimensional latent bottleneck & reconstruct the original input from the representation.
- Encoders: A sub network comprising 2D convolutional (Conv 2D) & downsampling (MaxPooling2D) layers that extracts spatial hierarchical features & reduces dimensionality.
- Bottleneck (Latent Space): The lowest-dimensional intermediate tensor representing compressed essential features while eliminating noise & redundancy.
- Decoder: A sub-network utilizing transposed convolutions (Conv2DTranspose) that upsamples the compressed latent representations back to the original input resolutions (28×28×1).
- Loss function: Binary Cross-Entropy is used to measure the pixel-wise difference between the input image & the reconstructed image.

Algorithm

1. Data Ingestion & Normalization: Load the MNIST handwritten





CONCLUSION:

The convolutional Autoencoder effectively learned compressed spatial representations to reconstruct handwritten digit images with minimal pixel-level error.

Date..14:.9:.26.....

Expt. No...11..... Page. No...29.....

digits dataset; scale pixel values to the [0,1] interval & reshape inputs to (28, 28, 1).

2. Design Encoder Network: Stack Conv2D layers with ReLU activations followed by MaxPooling2D layers to downsample spatial dimensions from 28×28×1 to a 4×4×8 bottleneck.

3. Design Decoder Network: Construct symmetric reconstruction stages using Conv2DTranspose layers to upsample the latent representation back to 28×28.

4. Output Layer: Apply a final Conv2D layer with a Sigmoid activation to bound output pixel values within [0,1]

5. Model Assembly & Compilation: Connect the encoder & decoder into an end-to-end model; compile using the Adam optimizer & Binary Cross-Entropy loss.

6. Self-Supervised Training: Train the autoencoder by passing training images as both the model inputs & targets (x=y).

7. Evaluation & Visual Comparision: Pass unseen test digits through the model & plot side-by-side visual comparisions of original versus reconstructed images.

Conclusion:

The convolutional Autoencoder learns spatial feature encodings without manual annotations. By compressing input digits into a compact latent bottleneck & reconstructing them with transposed convolutions, it preserves essential structural details while discarding redundant pixel information.

ASTRIX

Teacher's Signature.....

PRACTICAL-12

AIM:

To generate realistic synthetic handwritten digits from random noise using a Deep Convolutional Generative Adversarial Network.

AISHI DE. 2307053100 8

```
: Generative Adversarial Network (GAN). Problem Statement: Develop a GAN model to generate synthetic handwritten digit images.
Also, implement Generator and Discriminator, Train GAN model, Visualize generated images.

import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST data
(X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

# Normalize images to [-1, 1]
X_train = (X_train.astype('float32') - 127.5) / 127.5
X_train = np.expand_dims(X_train, axis=-1)

# Buffer and batch the dataset
BUFFER_SIZE = 60000
BATCH_SIZE = 256

train_dataset = tf.data.Dataset.from_tensor_slices(X_train)\
    .shuffle(BUFFER_SIZE)\
    .batch(BATCH_SIZE)

print(f"Dataset shape: {X_train.shape}")

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Dataset shape: (60000, 28, 28, 1)

LATENT_DIM = 100

# 1. Generator Architecture
def build_generator():
    model = models.Sequential(name="Generator")

    # Foundation for 7x7 image
    model.add(layers.Input(shape=(LATENT_DIM,)))
    model.add(layers.Dense(7 * 7 * 128))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Reshape((7, 7, 128)))

    # Upsample to 14x14
    model.add(layers.Conv2DTranspose(64, kernel_size=4, strides=2, padding='same'))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU(negative_slope=0.2))

    # Upsample to 28x28
    model.add(layers.Conv2DTranspose(1, kernel_size=4, strides=2, padding='same', activation='tanh'))
    return model

# 2. Discriminator Architecture
def build_discriminator():
    model = models.Sequential(name="Discriminator")

    model.add(layers.Input(shape=(28, 28, 1)))
    model.add(layers.Conv2D(64, kernel_size=4, strides=2, padding='same'))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, kernel_size=4, strides=2, padding='same'))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

generator = build_generator()
discriminator = build_discriminator()

generator.summary()
discriminator.summary()
```

Date 21:9:26

Expt. No. 12 Page. No. 30

PRACTICAL - 12

AIM:

To design, implement & train a Deep Convolutional Generative Adversarial Network (DCGAN) on the MNIST dataset to synthesize realistic handwritten digit images from random noise.

Theory:

- Generative Adversarial Networks (GANs): A framework where two neural networks compete in a zero sum game:
  - Generator (G): Maps a random latent noise vector $z \sim p_z(z)$ into synthetic data samples, attempting to fool the discriminator
  - Discriminator (D): A binary classifier that evaluates whether an input image comes from the true training distribution (1) or the generator (0).
- Adversarial Min-Max Objectives:

$$\min_G \max_D V(D,G) = E_{x \sim p_{data}(x)}[\log D(x)] + E_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

- Deep Convolutional GAN (DCGAN) Principles:
  - Transpose Convolutionals (Conv2DTranspose): Upsamples feature maps from spatial dimensions 7×7 to 14×14 to 28×28 in the generator.
  - Batch Normalization: Stabalizes training by normalizing layer outputs, preventing the generator from collapsing into a single mode (Mode Collapse).
  - LeakyReLU Activations: Prevents dying gradients during backpropagation in both the discriminator & generator.

dropout_3 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_3 (Dense)	(None, 1)	6,273

Total params: 130,561 (509.25 KB)
Trainable params: 130,561 (509.25 KB)
Non-trainable params: 0 (0.00 B)

```
# Optimizers
generator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4, beta_1=0.5)

# Fixed: BinaryCrossentropy with lowercase 'entropy'
cross_entropy = tf.keras.losses.BinaryCrossentropy()

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, LATENT_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))

    return gen_loss, disc_loss
```

```
EPOCHS = 7
seed = tf.random.normal([16, LATENT_DIM])

print("Training GAN...")
for epoch in range(EPOCHS):
    gen_loss_list = []
    disc_loss_list = []

    [illegible]

    for image_batch in train_dataset:
        if len(image_batch) == BATCH_SIZE:
            g_loss, d_loss = train_step(image_batch)
            gen_loss_list.append(g_loss)
            disc_loss_list.append(d_loss)

    print(f"Epoch {epoch + 1}/{EPOCHS} | Gen Loss: {tf.reduce_mean(gen_loss_list):.4f} | Disc Loss: {tf.reduce_mean(disc_loss_list):.4f}")

print("Training Complete!")
```

```
Training GAN...
Epoch 1/7 | Gen Loss: 0.8399 | Disc Loss: 1.0645
Epoch 2/7 | Gen Loss: 0.7895 | Disc Loss: 1.1760
Epoch 3/7 | Gen Loss: 0.7626 | Disc Loss: 1.2259
Epoch 4/7 | Gen Loss: 0.7388 | Disc Loss: 1.2588
Epoch 5/7 | Gen Loss: 0.7202 | Disc Loss: 1.2826
Epoch 6/7 | Gen Loss: 0.7060 | Disc Loss: 1.2999
Epoch 7/7 | Gen Loss: 0.6950 | Disc Loss: 1.3128
Training Complete!
```

```
# Generate images from fixed seed
predictions = generator(seed, training=False)

# Rescale from [-1, 1] back to [0, 1] for visualization
predictions = (predictions + 1.0) / 2.0

plt.figure(figsize=(4, 4))
for i in range(predictions.shape[0]):
    plt.subplot(4, 4, i + 1)
    plt.imshow(predictions[i, :, :, 0], cmap='gray')
```

Date...21:9:26.....

Expt. No..12..... Page. No...31.....

Algorithm:

1. Data Normalization:
Load MNIST handwritten digits & normalize pixel values to [-1, 1] to align with the generator's tanh activation.

2. Design Generator:
Construct a network starting with a Dense layer to project latent noise vectors (dim = 100), followed by Conv2DTranspose, BatchNormalization & LeakyReLU & Dropout (0.3) layers ending with a 28×28×1 tanh output.

3. Design Descriminator:
Construct a convolutional classifier using strided Conv2D, LeakyReLU & Dropout (0.3) layers ending with a 1-unit sigmoid output.

4. Define Loss & Optimization:
Use Binary Cross Entropy loss for both models with Adam optimizers ($lr = 2 \times 10^{-4}$, $\beta_1 = 0.5$).

5. Adversarial Training Loop:
- Sample random Gaussian noise & generate fake images
- Compute Discriminator loss on real & fake batches; update Discriminator weights.
- Compute Generator loss based on the Discriminator's misclassification of fake images; update Generator weights.

6. Inference & Visualization:
Pass a fixed latent noise vector to the trained Generator, rescale pixel values to [0, 1] & plot synthetic digit grids.

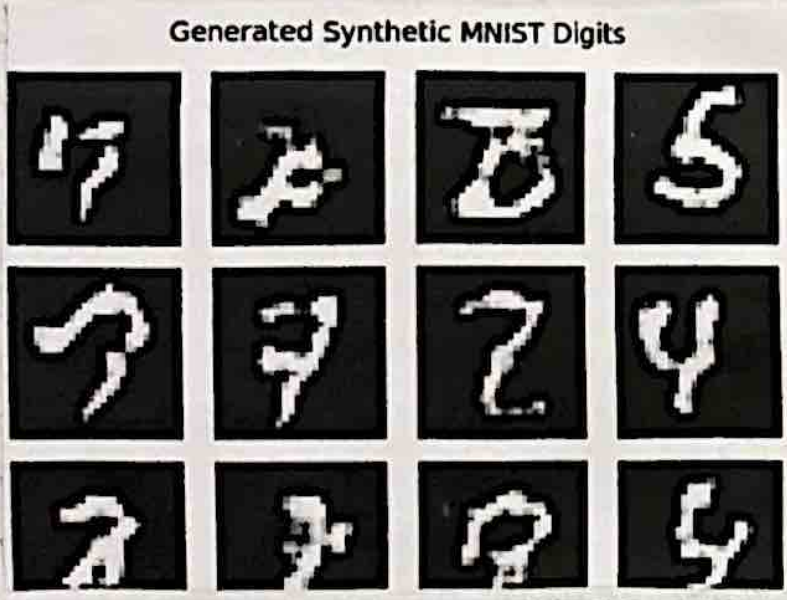
ASTRIX

Teacher's Signature.....

Model: "Generator"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 6272)	633,472
batch_normalization (BatchNormalization)	(None, 6272)	25,088
leaky_re_lu (LeakyReLU)	(None, 6272)	0
reshape (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	131,136
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_1 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 1)	1,025

Total params: 790,977 (3.02 MB)
Trainable params: 778,305 (2.97 MB)
Non-trainable params: 12,672 (49.50 KB)
Model: "Discriminator"



Conclusion:

The DCGAN succesfully learned the MNIST image distribution through adversarial training, producing realistic & novel handwritten digit images from latent noise vectors.

Date 21.9.26

Expt. No. 12 Page. No. 32

Conclusion :-

The DCGAN learns the underlying spatial data distribution of the MNIST dataset through adversarial competition. By balancing the optimization of the generator & discriminator, the model synthesizes novel, coherent handwritten digits from random latent noise vectors.

Teacher's Signature.....